

FATEC

Faculdade de Tecnologia

Grupo: Hello World

Alicia Silva Dias

Bruna Rodrigues Gomes

Bruno Berval Moreira de Godoi

Pedro Enrique de Jesus Freitas

Nicolas Kaue da Silva

Ryan Pedro Tinel de Andrade

Suelen Souza de Castro

GERENCIAR O FLUXO DE ATENDIMENTO

Banco de Dados Não Relacional — MongoDB

Jacareí / São Paulo · 2026

Introdução

O presente projeto tem como objetivo desenvolver um banco de dados não relacional utilizando MongoDB para representar o sistema de gestão de leads da empresa 1000 Valle Multimarcas.

A proposta foi baseada em um cenário real de atendimento comercial, no qual clientes interessados em veículos entram em contato com a empresa por diferentes canais, como WhatsApp, Instagram, telefone, formulários digitais e atendimento presencial.

Durante o desenvolvimento foram criadas collections para armazenar informações sobre clientes, usuários, lojas, leads, negociações e logs do sistema. Foram aplicados conceitos fundamentais de modelagem não relacional, utilizando embedding para o histórico das negociações e referencing através de ObjectId para relacionar os documentos entre as collections.

Também foram realizadas consultas utilizando operadores do MongoDB, como \$and, \$or, \$gt, \$lt e \$exists, além de aggregations para geração de indicadores gerenciais, como leads por origem, status, atendente e importância.

O projeto permitiu compreender na prática o funcionamento do MongoDB e as vantagens dos bancos de dados não relacionais em sistemas modernos, oferecendo maior flexibilidade, organização e facilidade no armazenamento e análise das informações.

Estrutura do Banco de Dados

O banco de dados foi estruturado utilizando o modelo não relacional do MongoDB, dividido em collections responsáveis pelo armazenamento das informações do sistema de gestão de leads da empresa 1000 Valle Multimarcas.

```
_MONGOSH
> use valle_multimarcas
< switched to db valle_multimarcas
> db.createCollection("clientes")

db.createCollection("lojas")

db.createCollection("usuarios")

db.createCollection("leads")

db.createCollection("negociacoes")

db.createCollection("logs")
< { ok: 1 }
> show collections
< clientes
leads
logs
lojas
negociacoes
usuarios
valle_multimarcas>
```

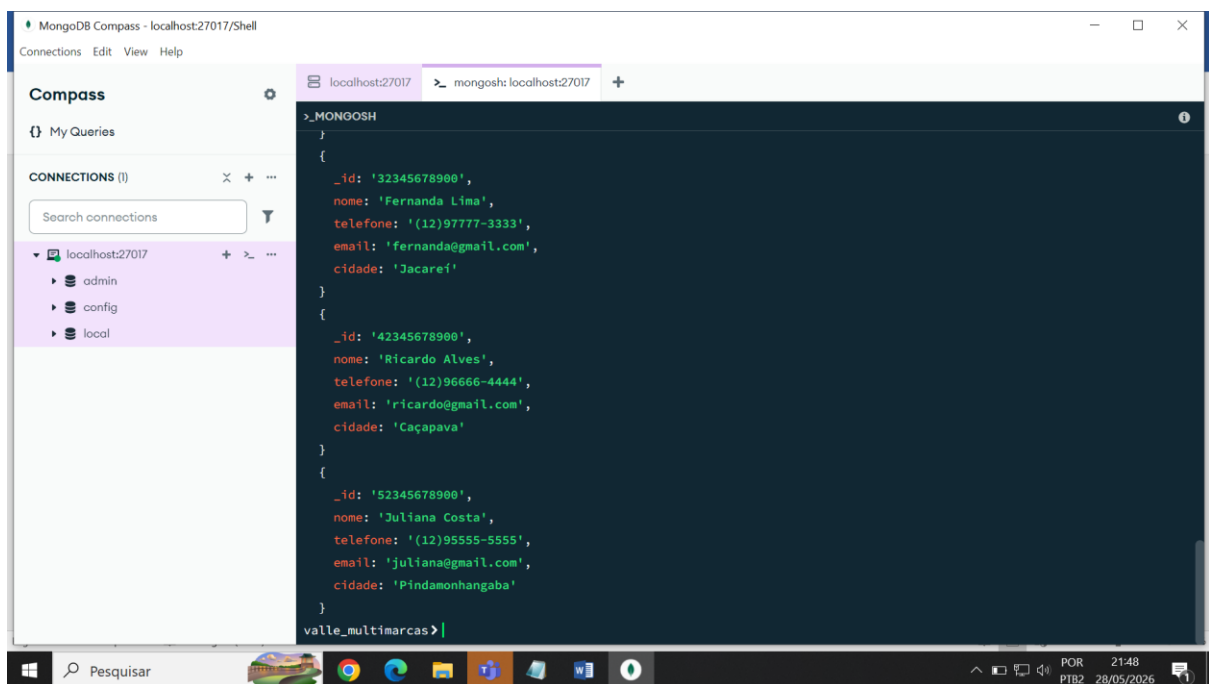
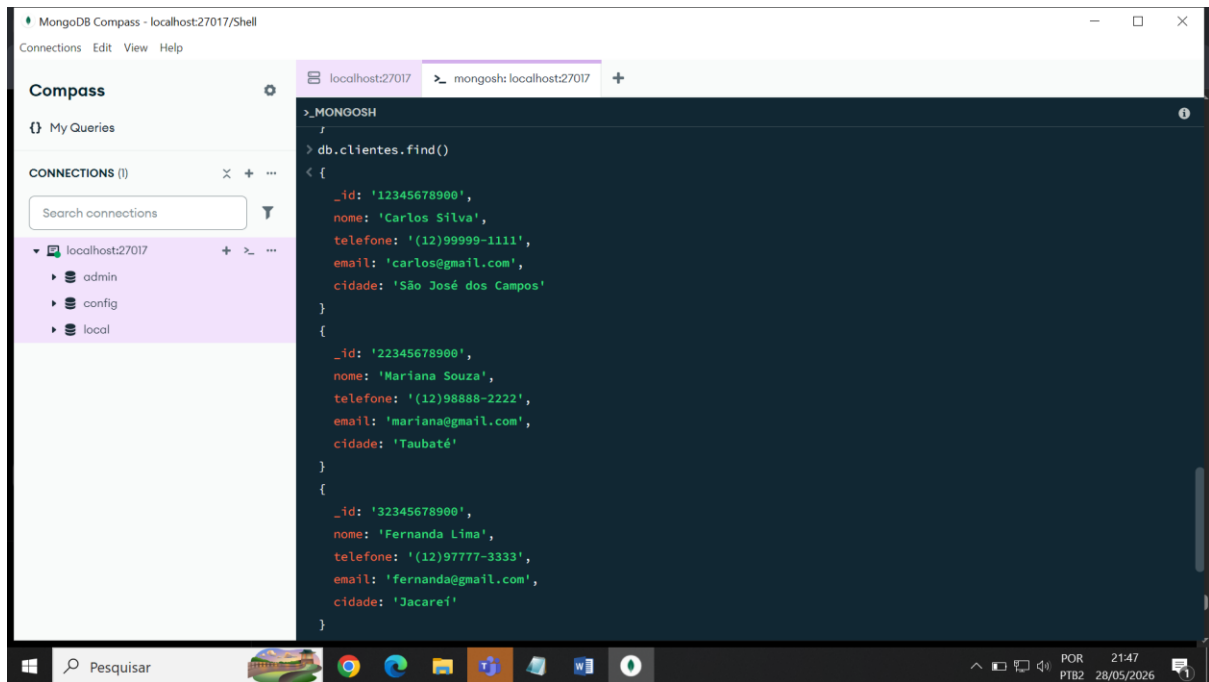
db.clients — Clientes

A collection clientes foi criada para armazenar os dados pessoais dos clientes interessados nos veículos. Cada documento contém as seguintes informações:

- nome — nome completo do cliente
- telefone — número de contato
- email — endereço de e-mail
- cidade — cidade de residência
- CPF — documento de identificação

Exemplo de documento inserido na collection:

```
db.clientes.insertOne({
  _id: '12345678900',
  nome: 'Carlos Silva',
  telefone: '(12)99999-1111',
  email: 'carlos@gmail.com',
  cidade: 'São José dos Campos'
})
```



db.lojas — Lojas

A collection lojas foi utilizada para representar as unidades da empresa 1000 Valle Multimarcas. Cada documento armazena:

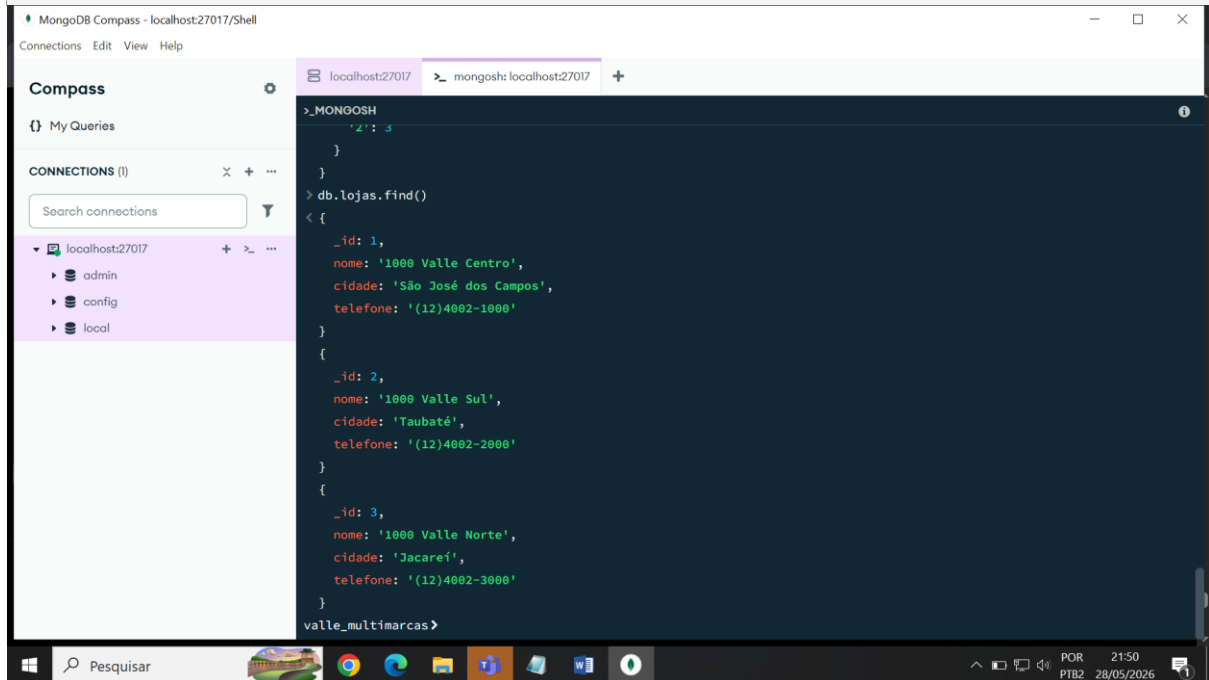
- nome — nome da unidade
- cidade — cidade onde a loja está localizada
- telefone — telefone de contato da unidade

Foram cadastradas três unidades no sistema:

```
// Unidade 1 - Centro
{ _id: 1, nome: '1000 Valle Centro',
  cidade: 'São José dos Campos', telefone: '(12)4002-1000' }

// Unidade 2 - Sul
{ _id: 2, nome: '1000 Valle Sul',
  cidade: 'Taubaté', telefone: '(12)4002-2000' }

// Unidade 3 - Norte
{ _id: 3, nome: '1000 Valle Norte',
  cidade: 'Jacareí', telefone: '(12)4002-3000' }
```



db.usuarios — Usuários do Sistema

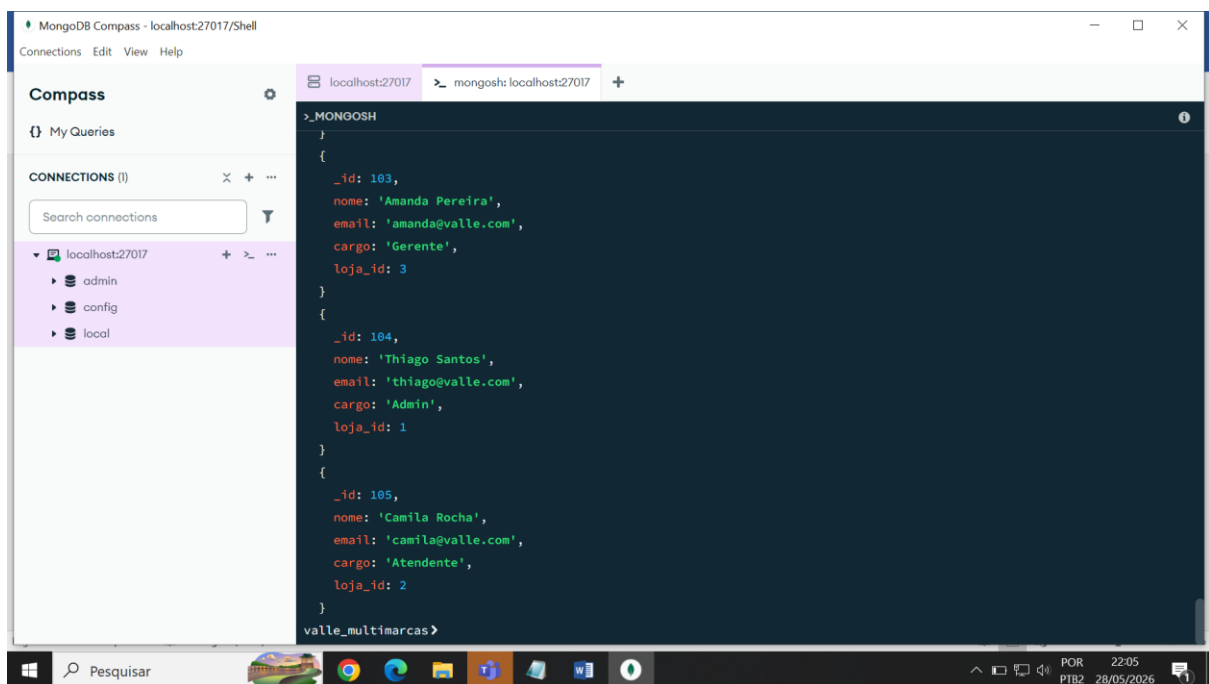
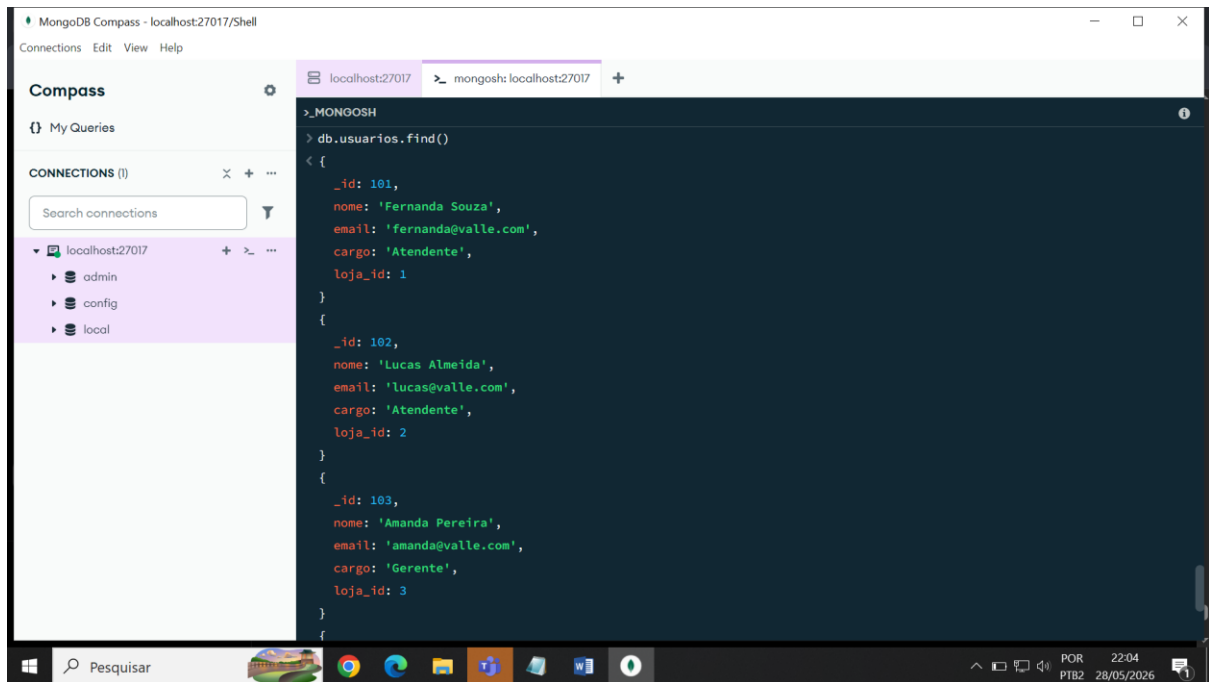
A collection usuarios armazena os colaboradores que utilizam o sistema, incluindo atendentes, gerentes e administradores. Cada documento contém o relacionamento com a loja onde o usuário atua, feito através de referencing pelo campo loja_id.

Campos de cada documento:

- nome — nome do usuário
- email — e-mail corporativo
- cargo — função (Atendente, Gerente, Administrador)
- loja_id — referência à loja onde o usuário trabalha

```
{ _id: 101, nome: 'Fernanda Sousa',
  email: 'fernanda@valle.com', cargo: 'Atendente', loja_id: 1 }

{ _id: 103, nome: 'Amanda Pereira',
  email: 'amanda@valle.com', cargo: 'Gerente', loja_id: 3 }
```



db.leads — Leads

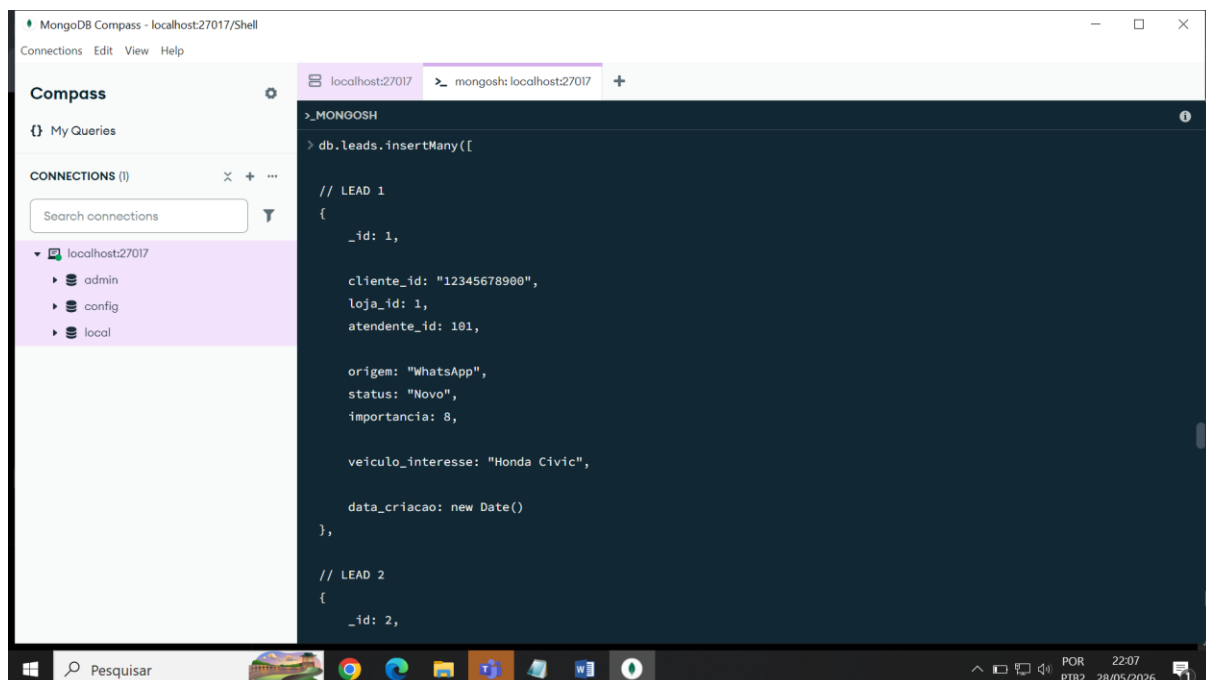
A collection leads registra os potenciais clientes que demonstraram interesse em adquirir um veículo. É a collection central do sistema, conectando clientes, lojas e atendentes por meio de ObjectId (referencing).

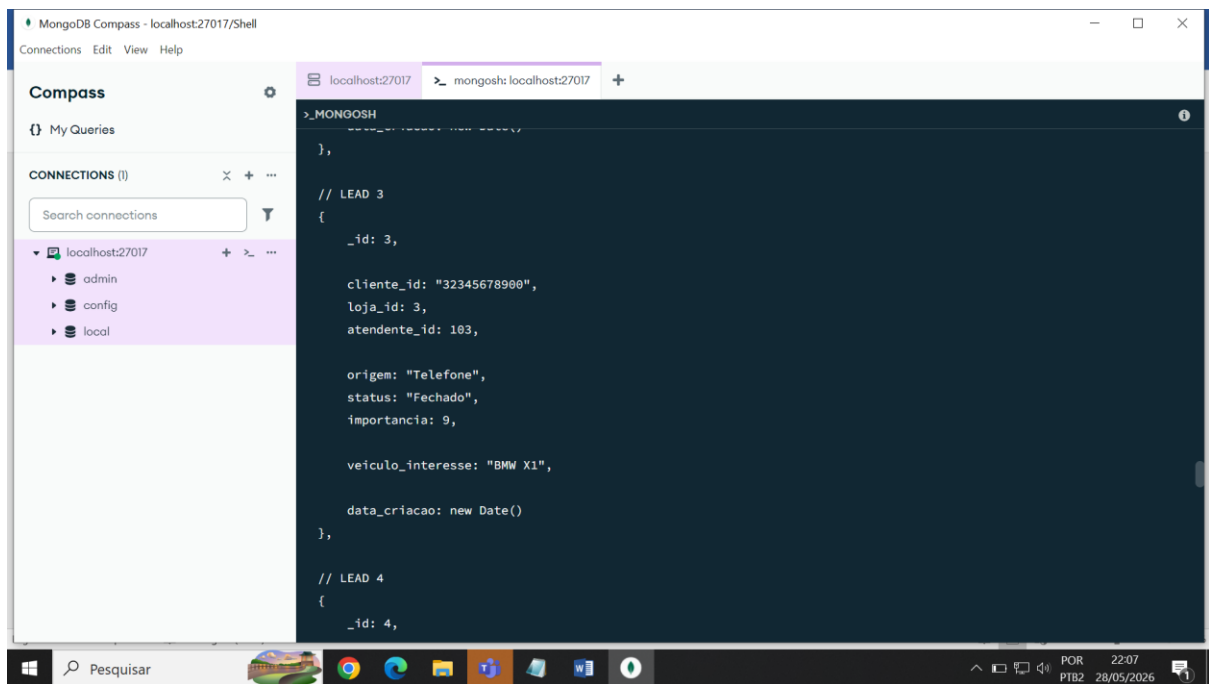
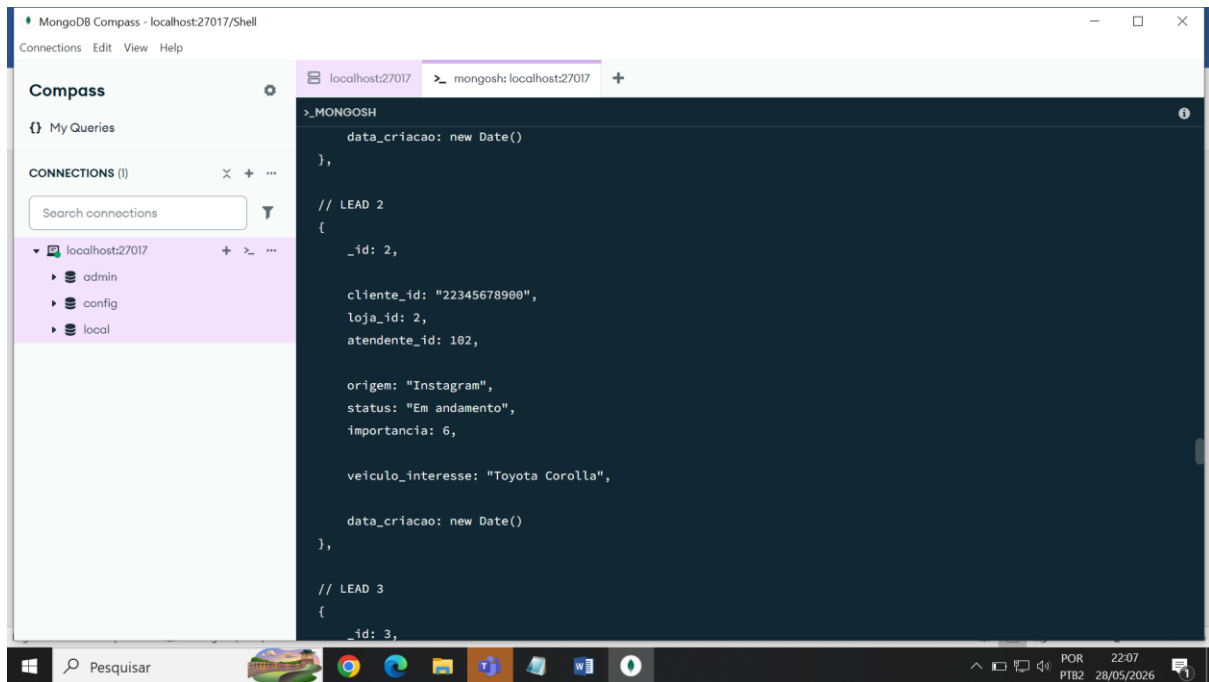
Campos de cada documento:

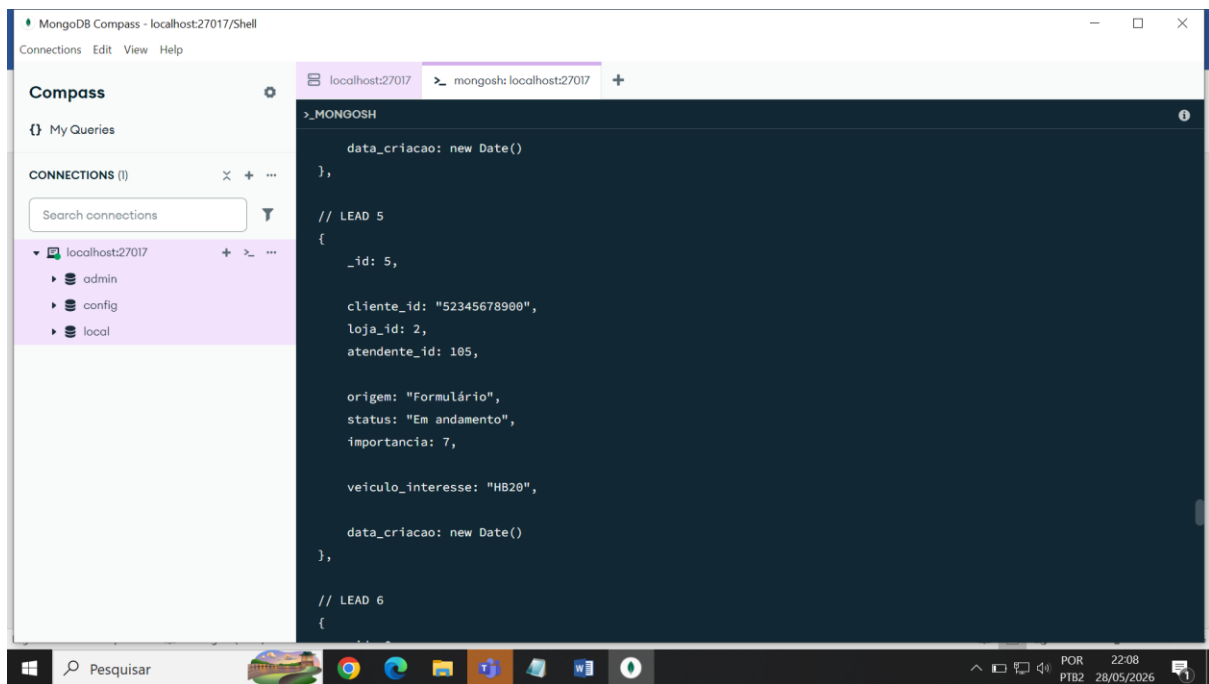
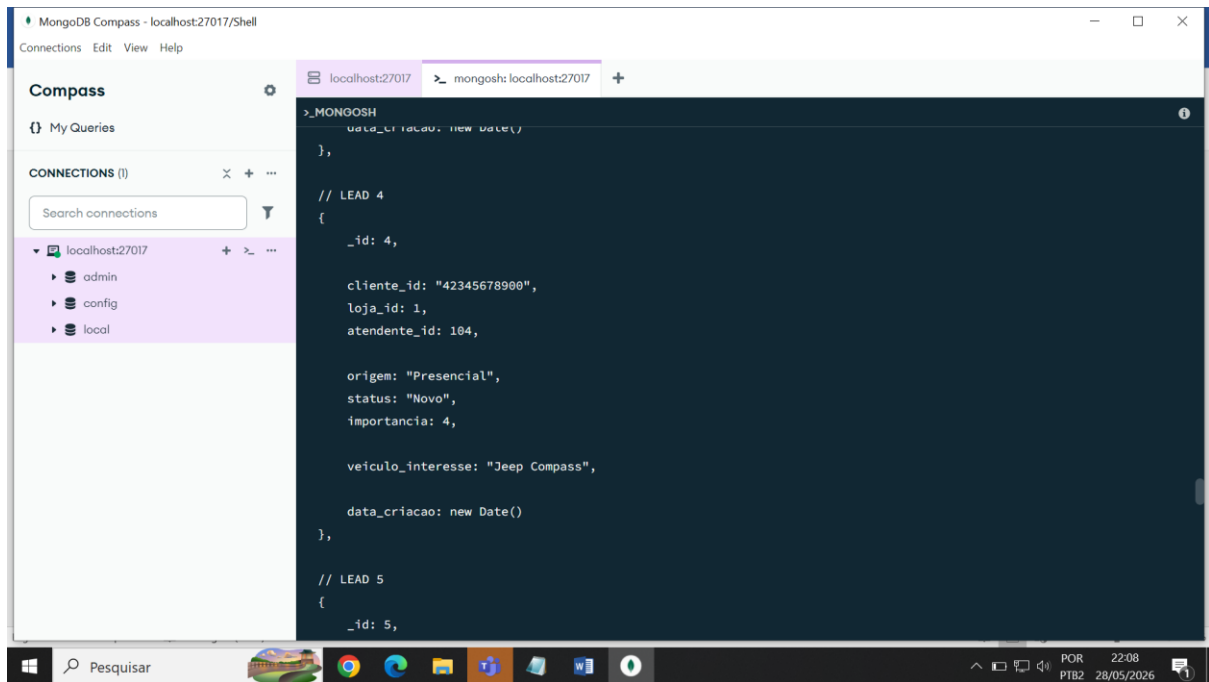
- cliente_id — referência ao cliente (ObjectId)
- loja_id — referência à loja de atendimento

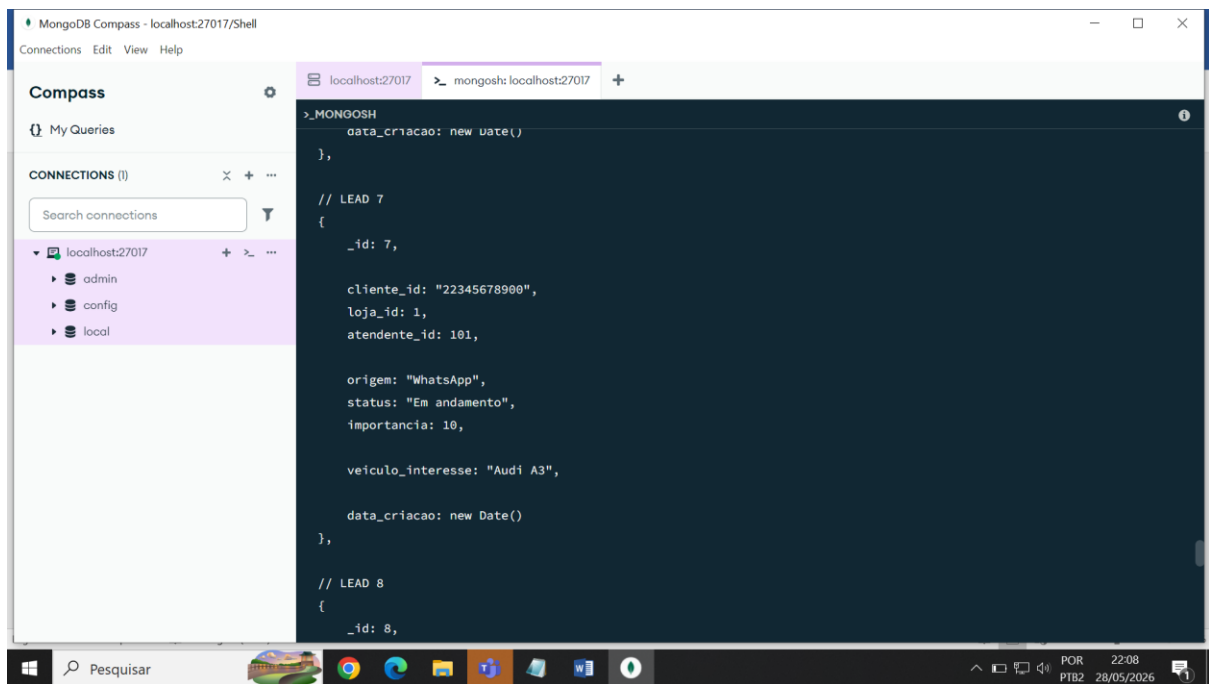
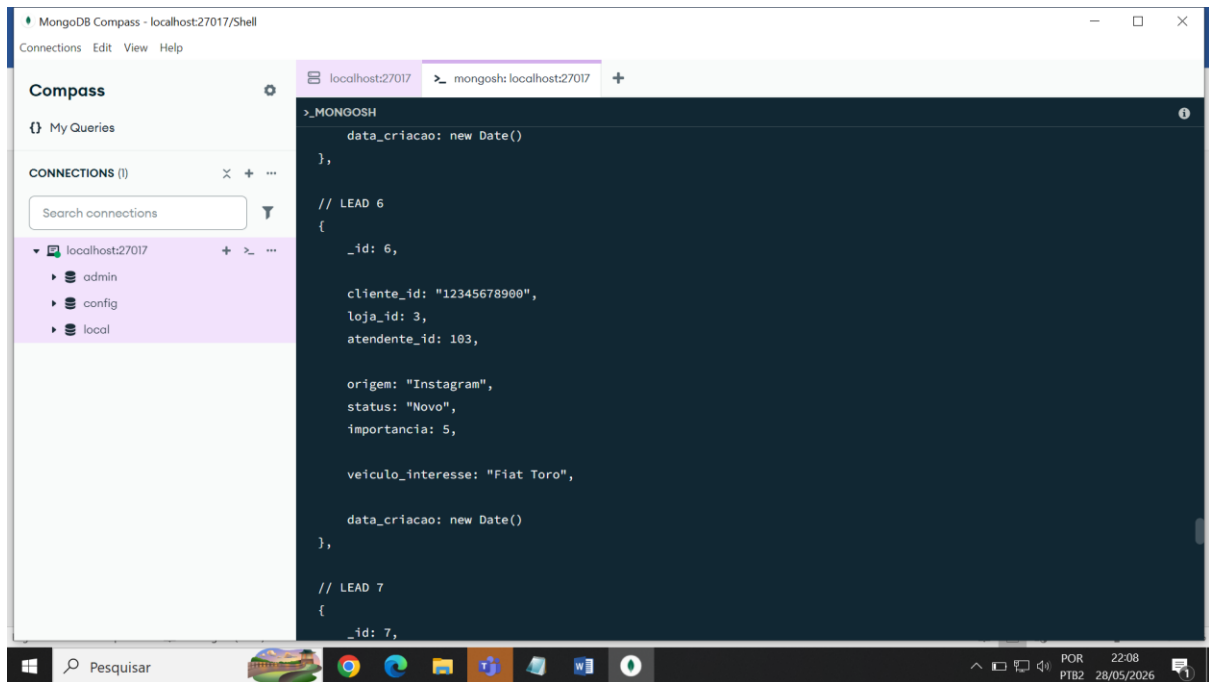
- `atendente_id` — referência ao atendente responsável
- `origem` — canal de entrada do lead (WhatsApp, Instagram, Telefone, Presencial, Formulário)
- `status` — situação atual (Novo, Em andamento, Fechado)
- `importancia` — nível de prioridade (1 a 10)
- `veiculo_interesse` — modelo de veículo de interesse
- `data_criacao` — data e hora do registro

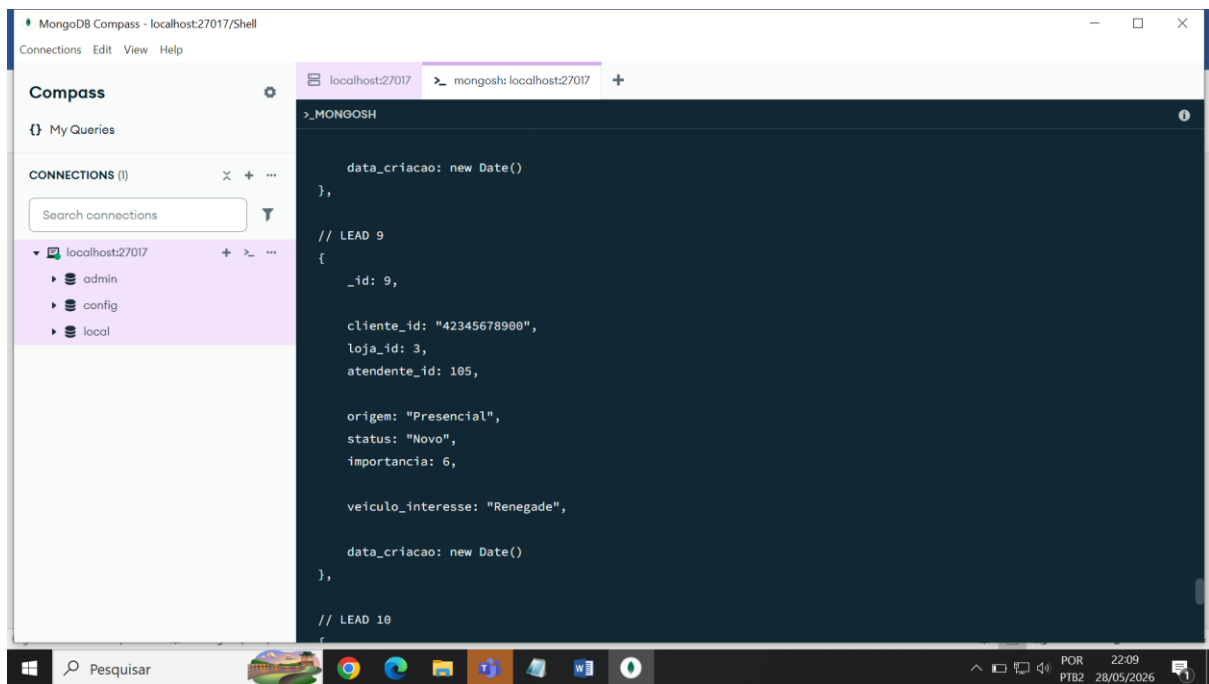
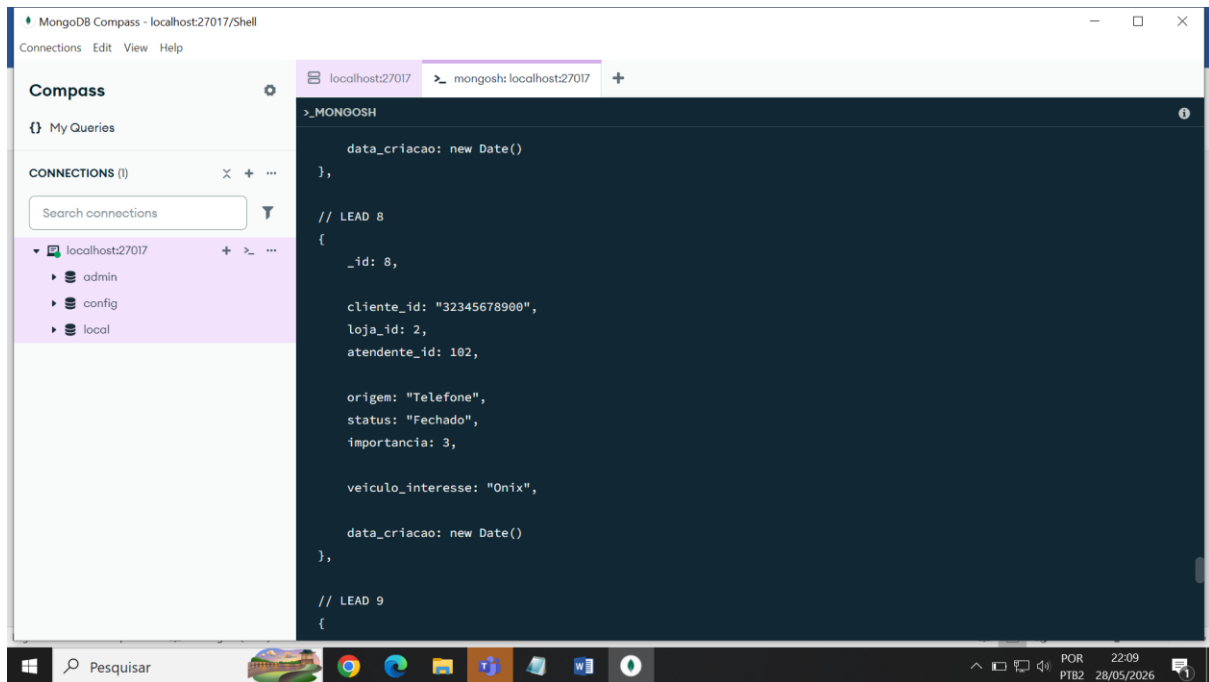
```
{ _id: 1,
  cliente_id: '12345678900', loja_id: 1, atendente_id: 101,
  origem: 'WhatsApp', status: 'Novo',
  importancia: 8, veiculo_interesse: 'Honda Civic',
  data_criacao: ISODate('2026-05-25') }
```

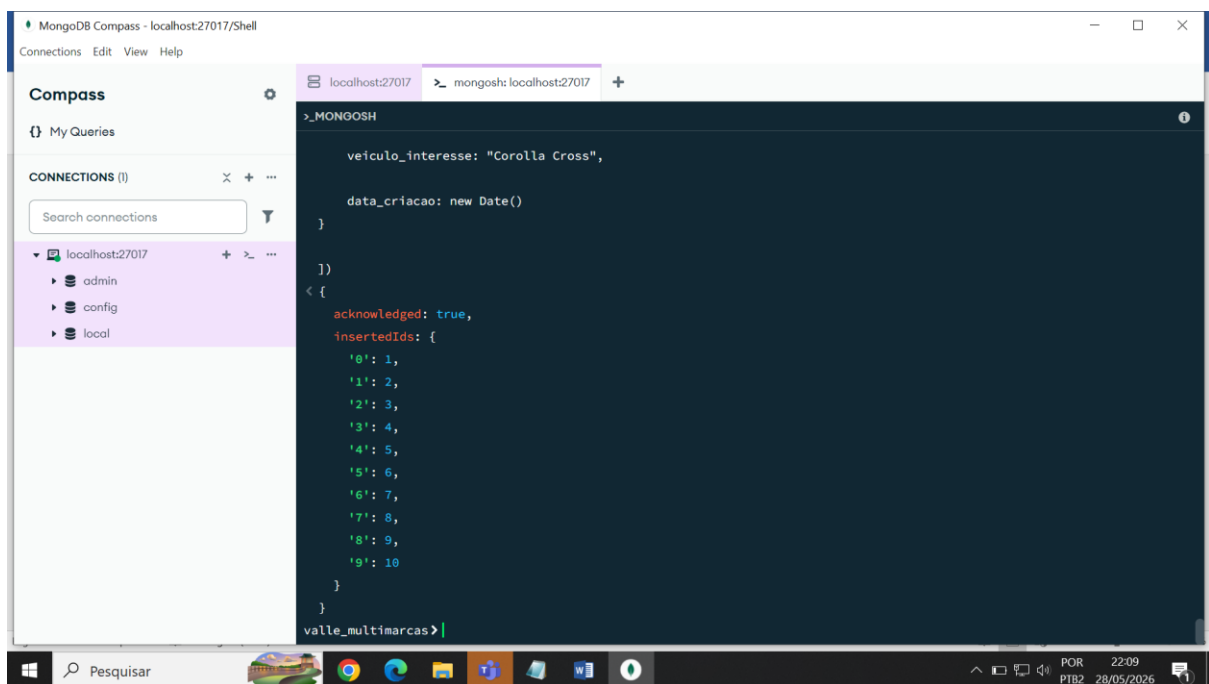
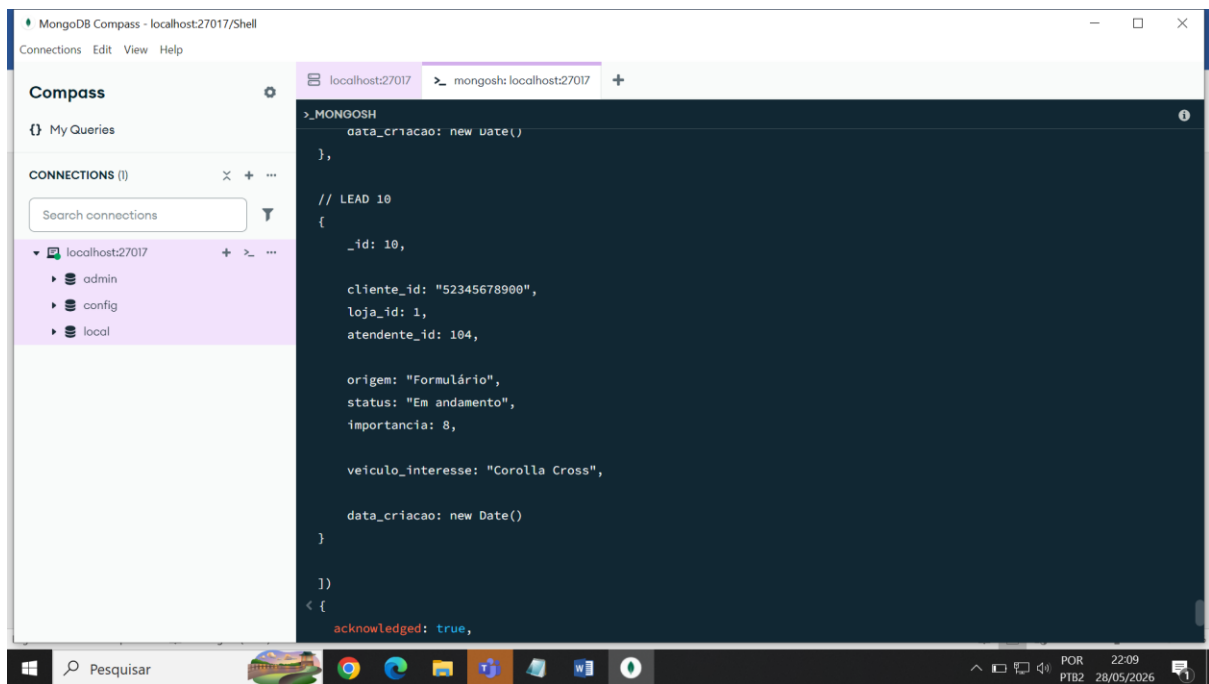












db.negociacoes — Negociações

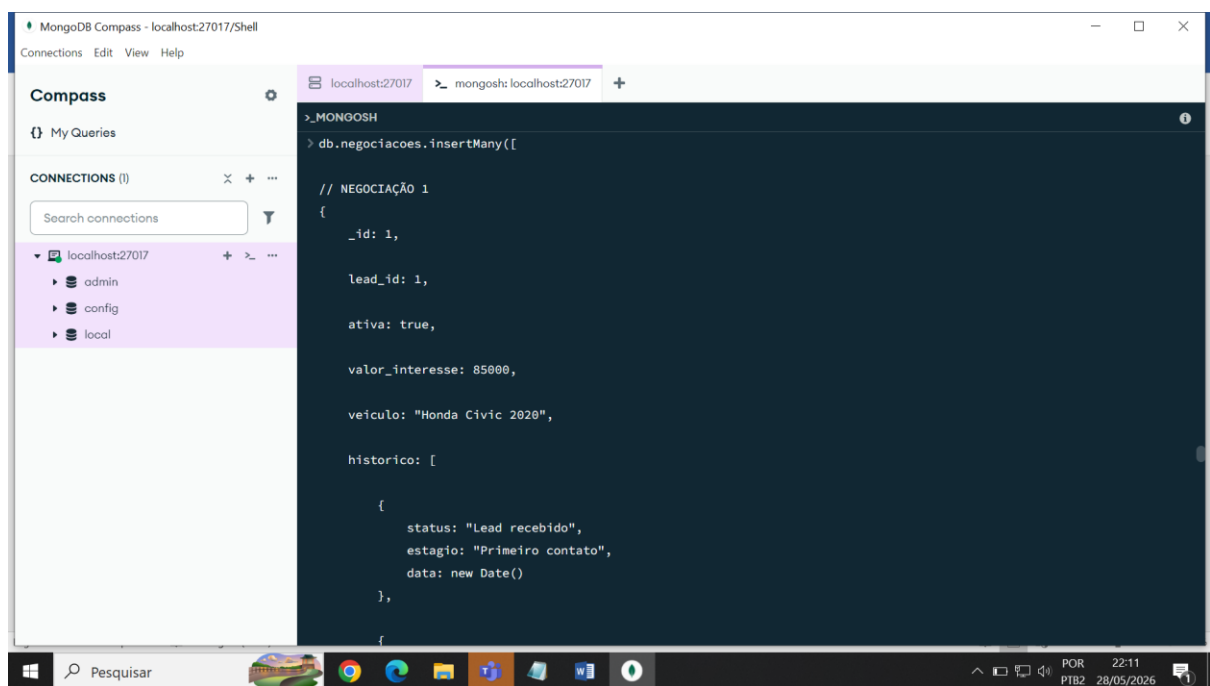
A collection negociacoes utiliza embedding no campo histórico, permitindo armazenar todas as etapas da negociação dentro do próprio documento. Isso evita consultas adicionais ao banco e mantém o histórico completo de forma organizada e eficiente.

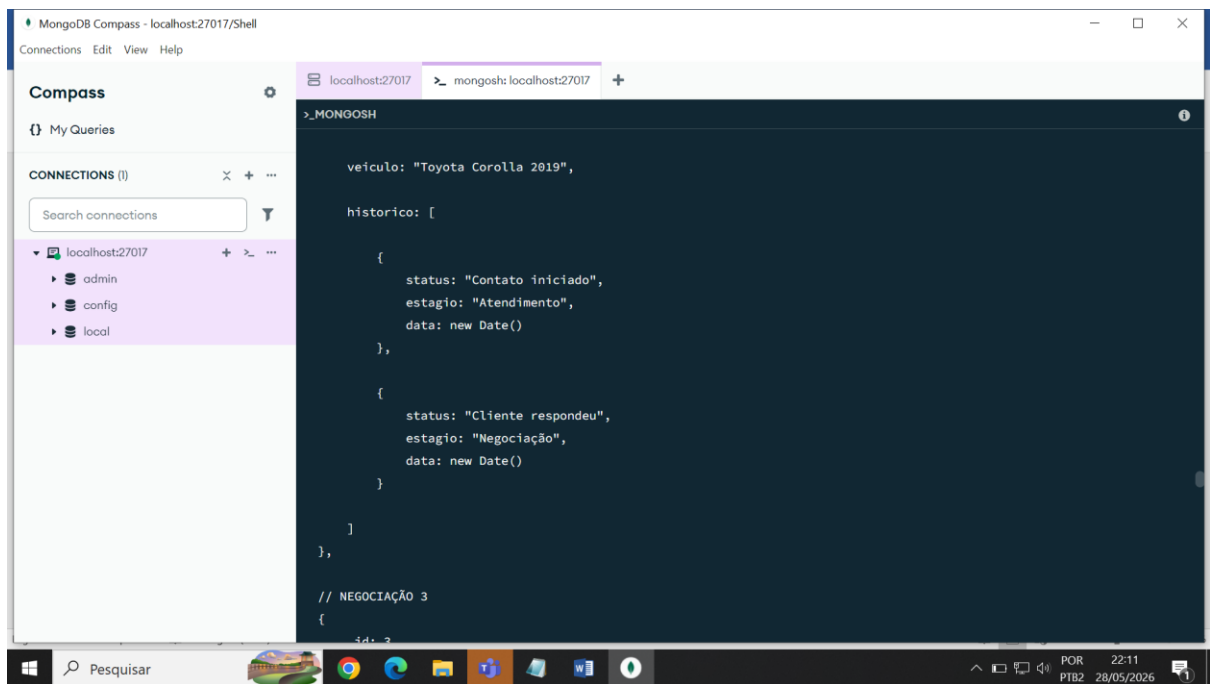
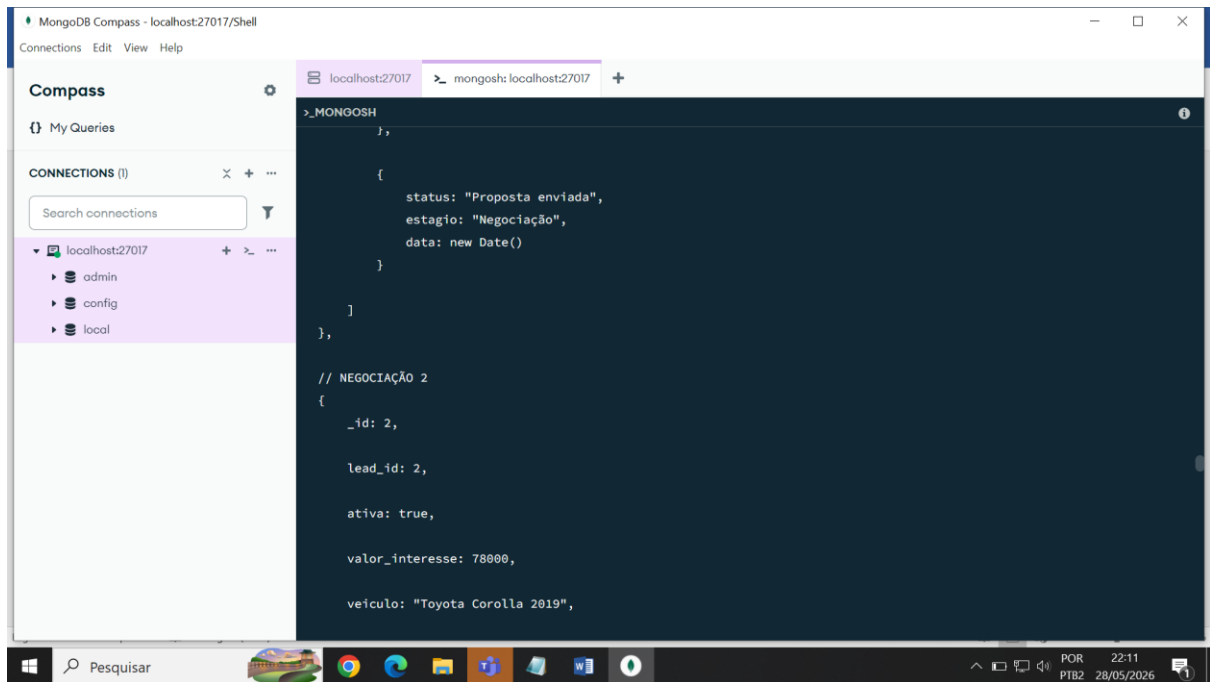
Campos principais:

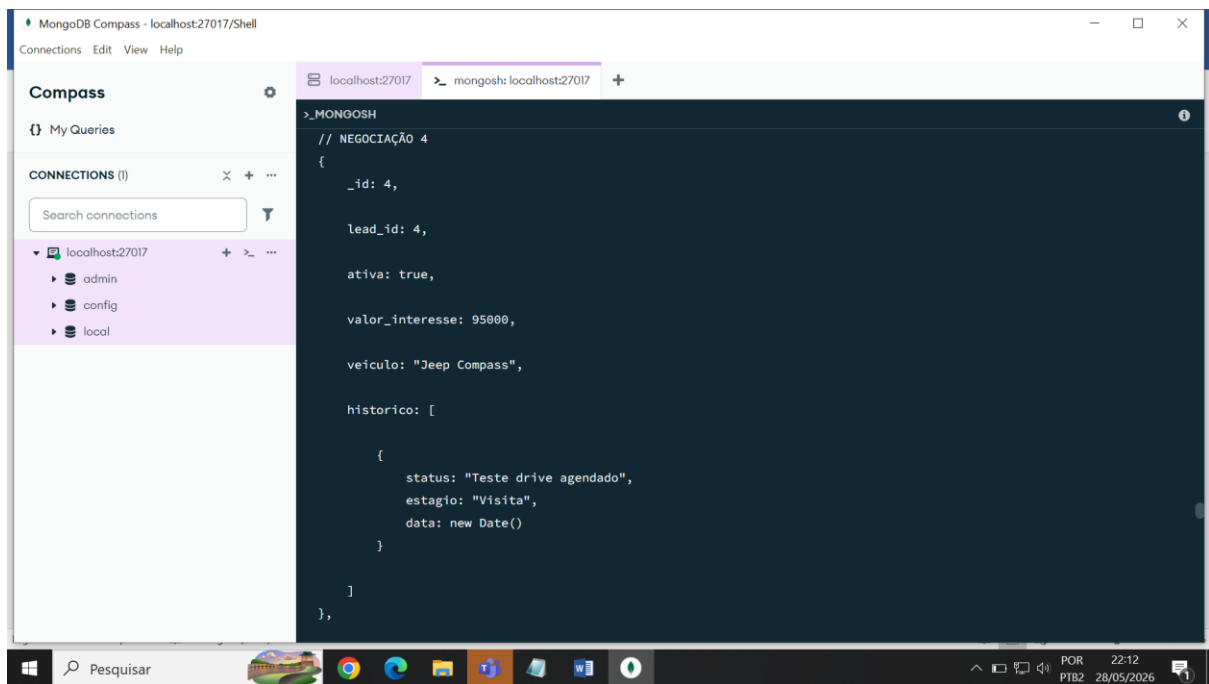
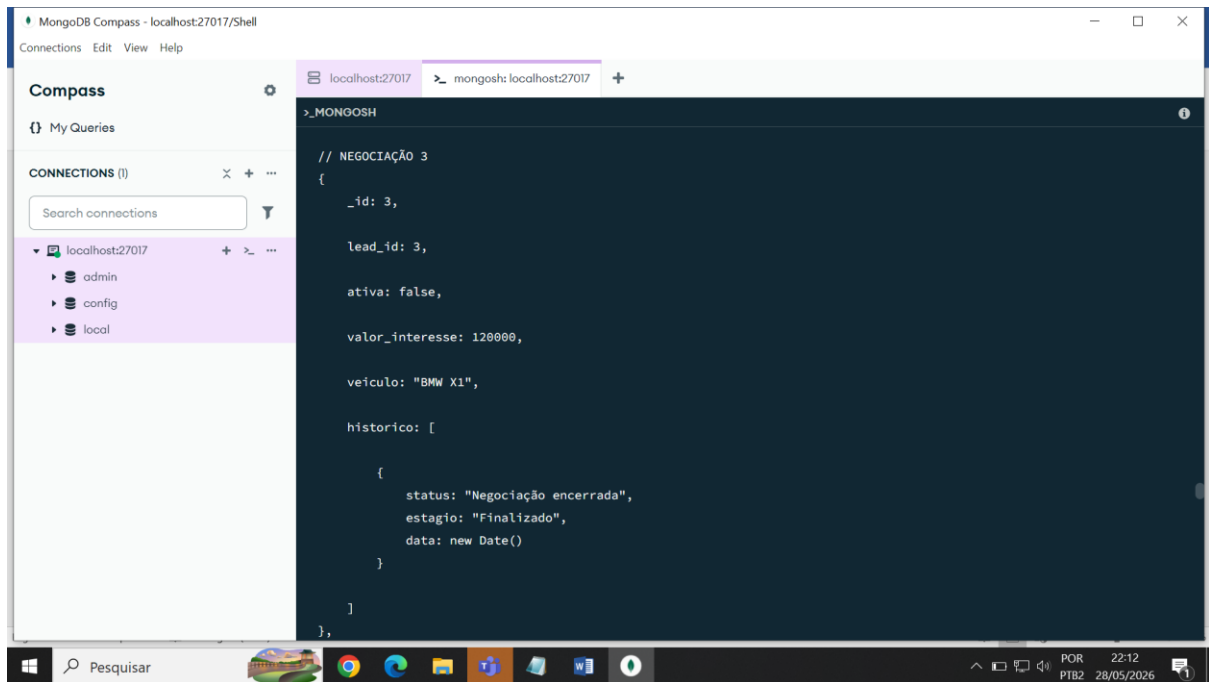
- lead_id — referência ao lead relacionado
- ativa — indica se a negociação está em andamento (true/false)

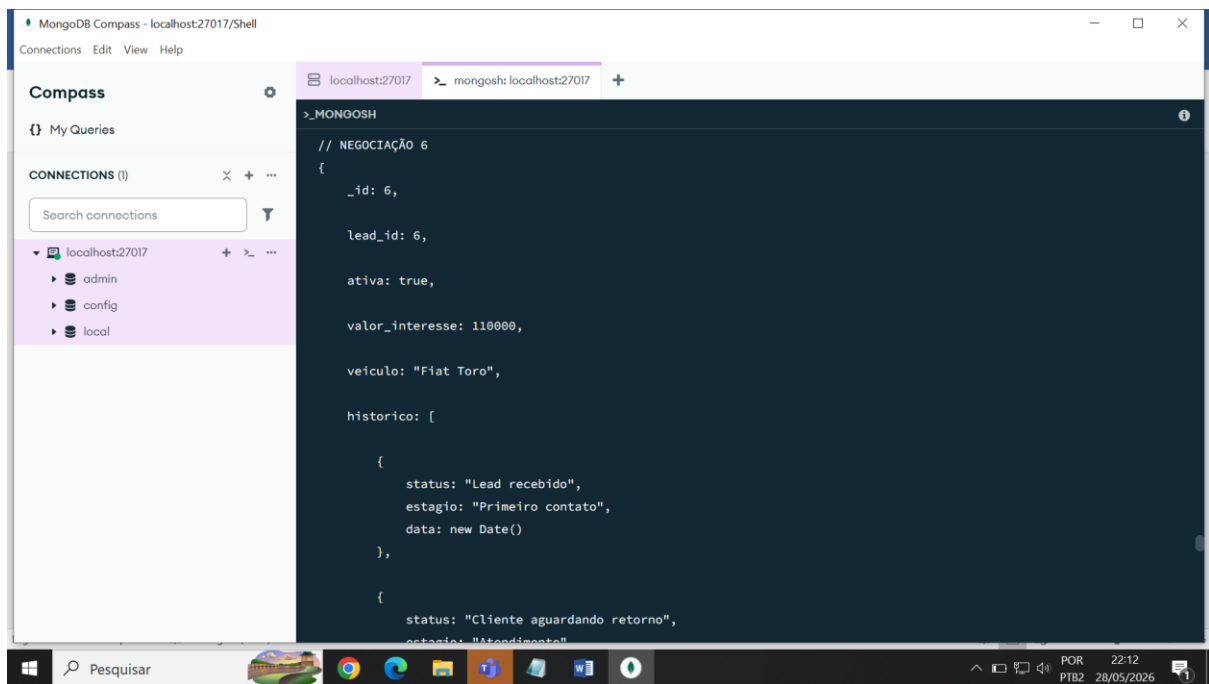
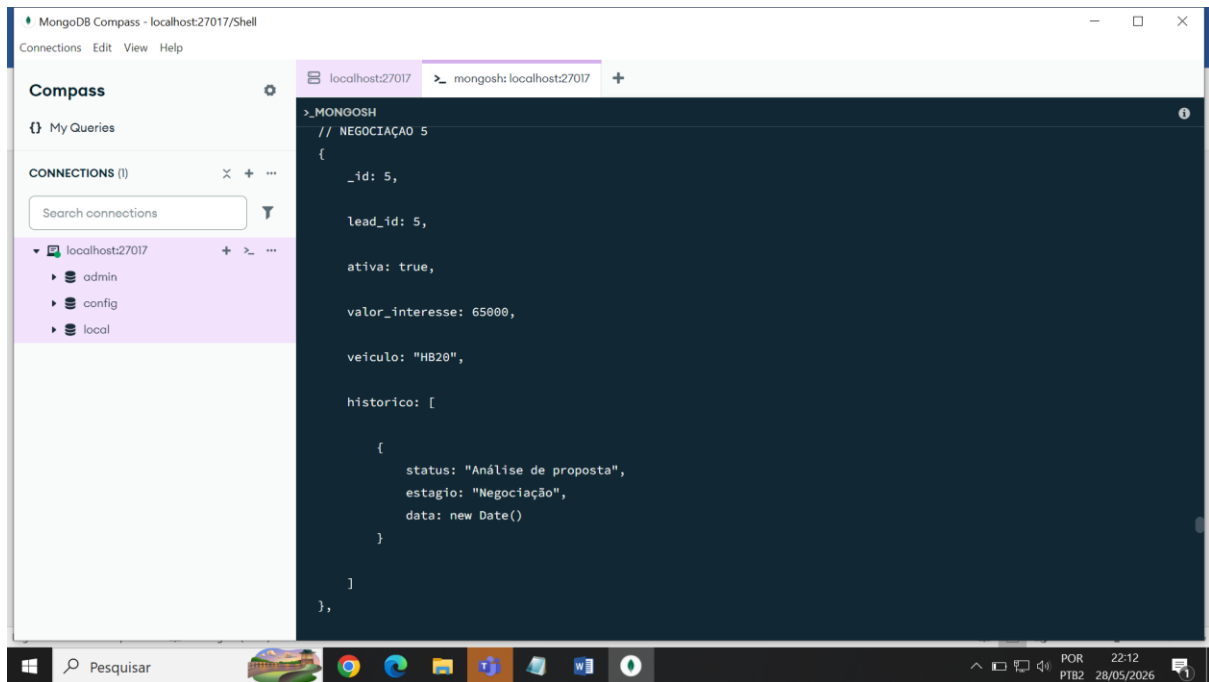
- valor_interesse — valor estimado do veículo
- veiculo — modelo em negociação
- historico — array de objetos com status, estágio e data de cada etapa

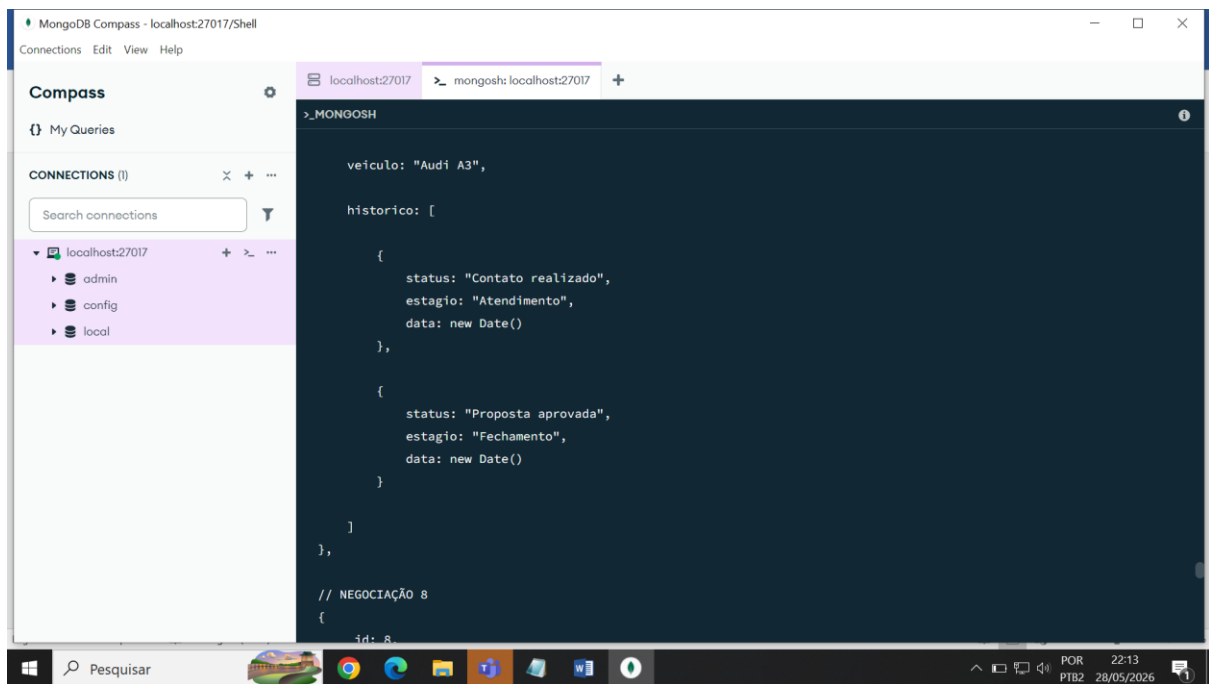
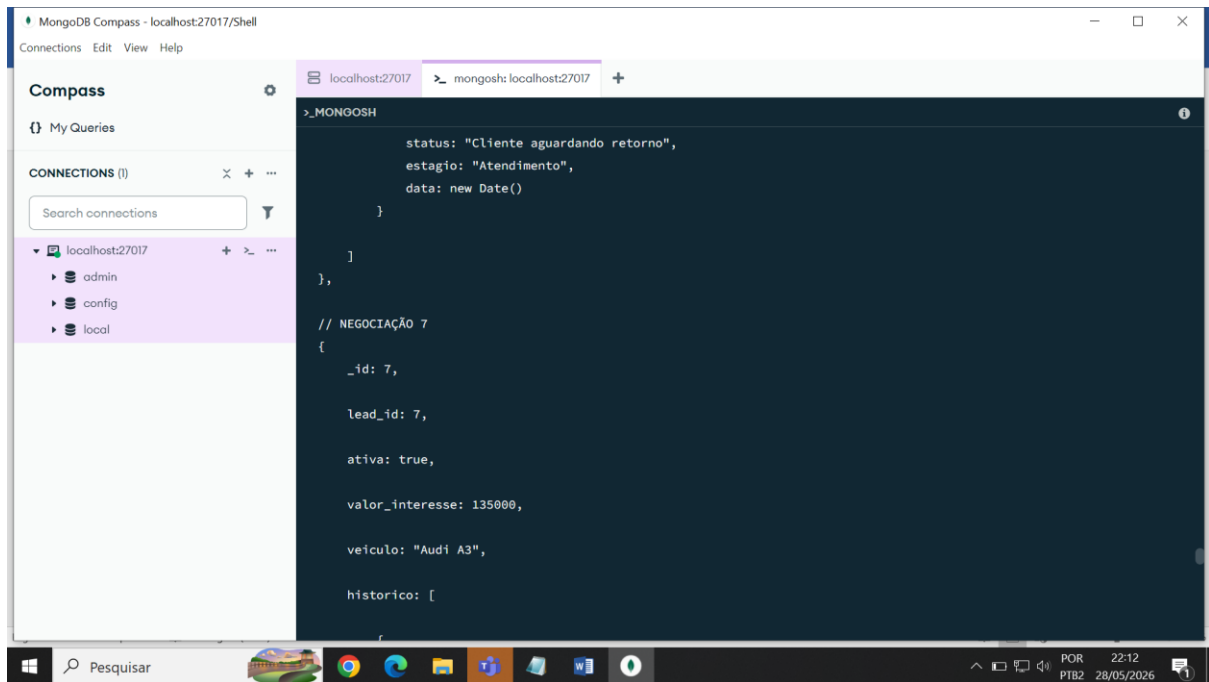
```
{ _id: 1, lead_id: 1, ativa: true,  
  valor_interesse: 85000, veiculo: 'Honda Civic 2020',  
  historico: [  
    { status: 'Lead recebido', estagio: 'Primeiro contato',  
      data: ISODate('2026-05-26') },  
    { status: 'Proposta enviada', estagio: 'Negociação',  
      data: ISODate('2026-05-26') }  
  ]  
}
```

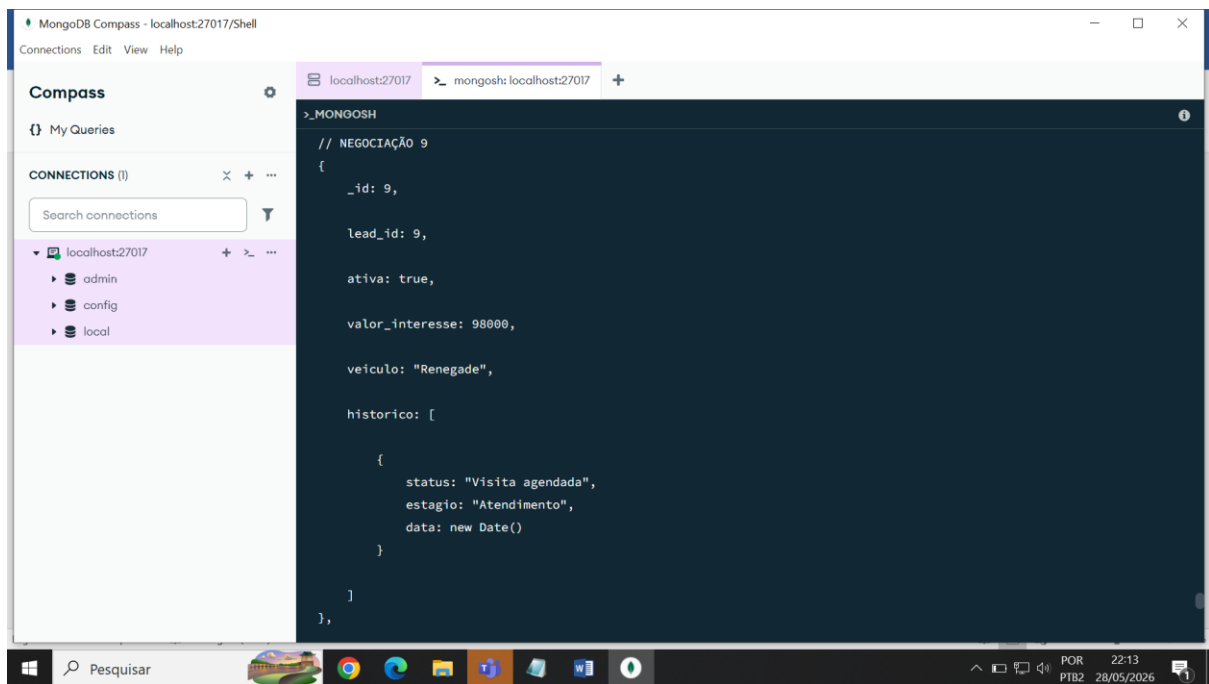
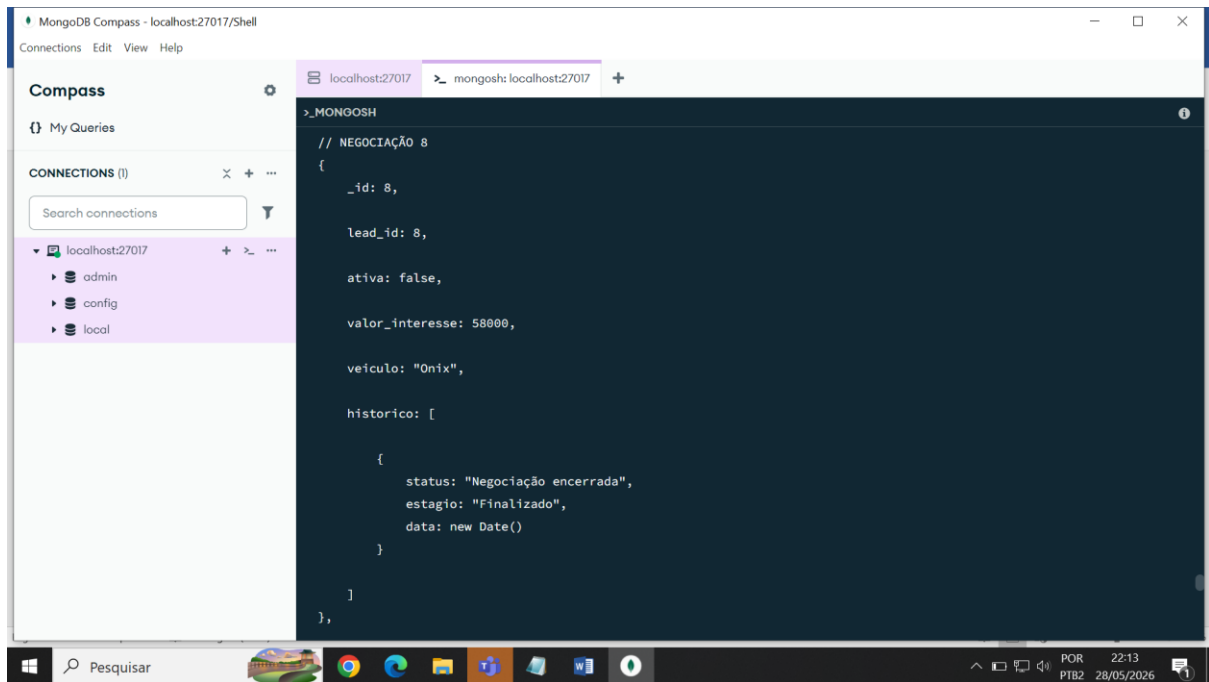


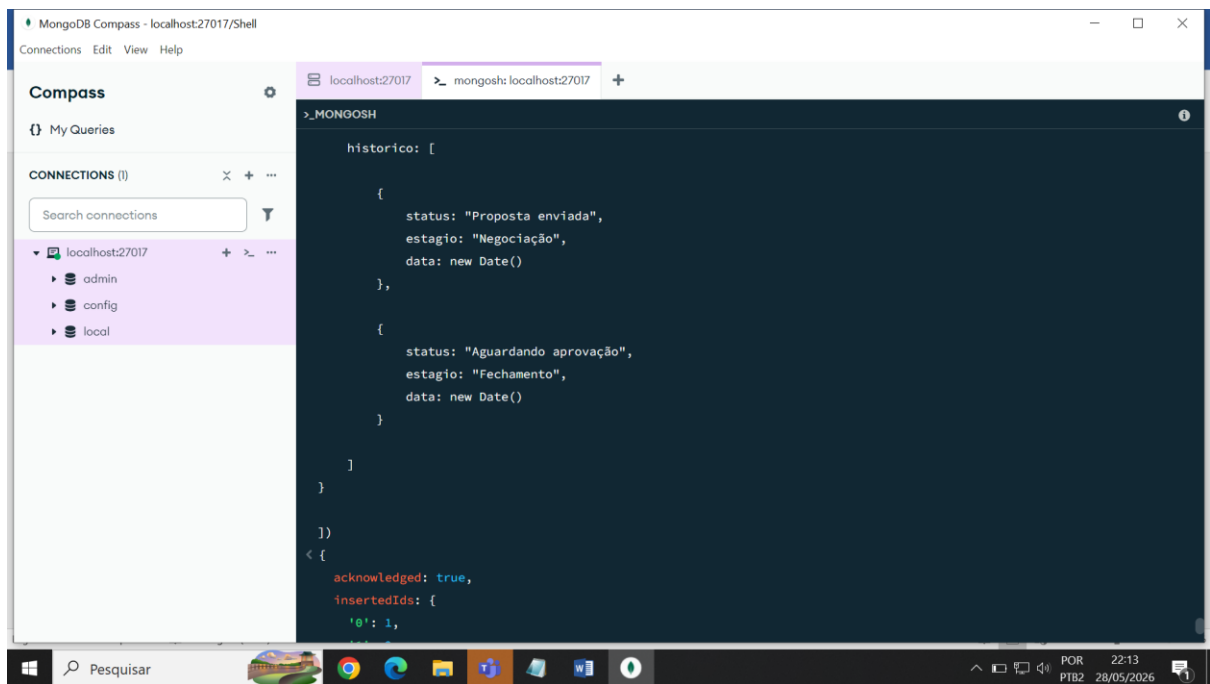
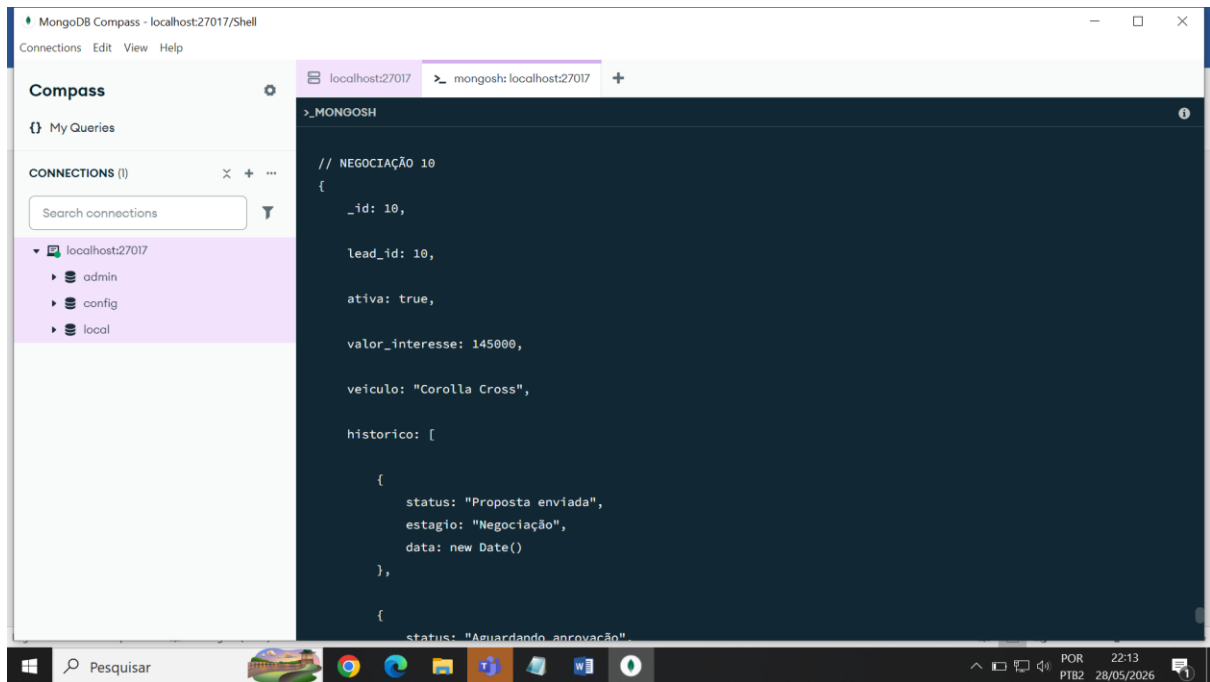


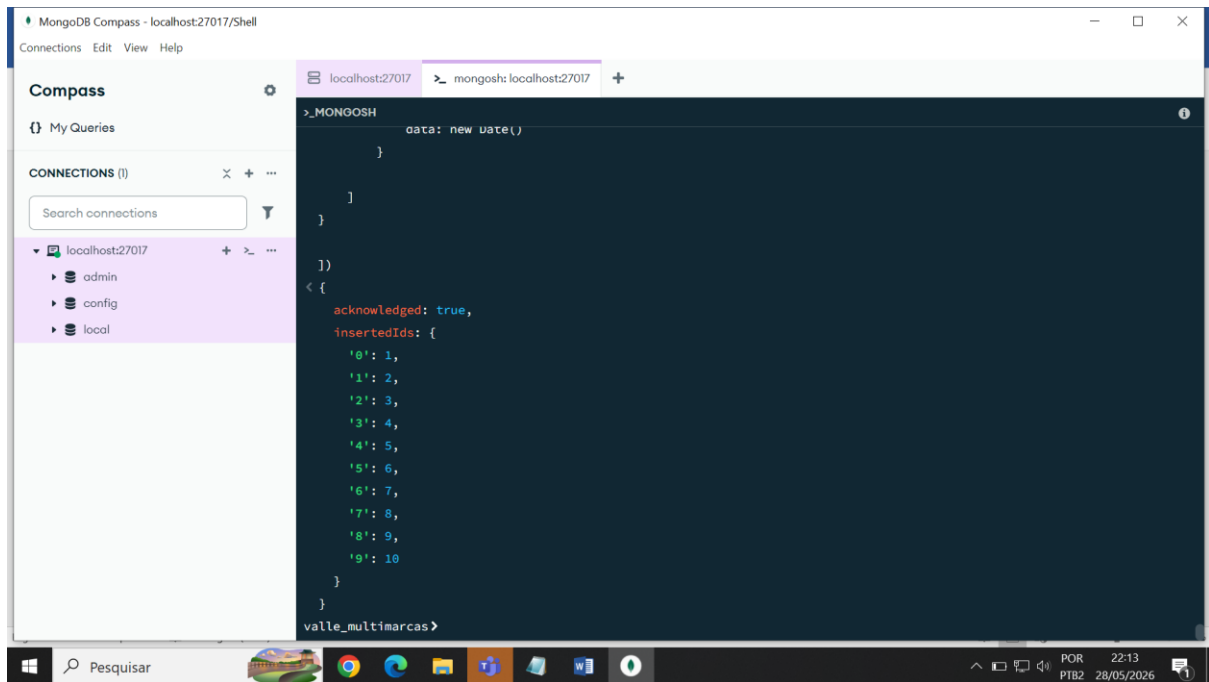












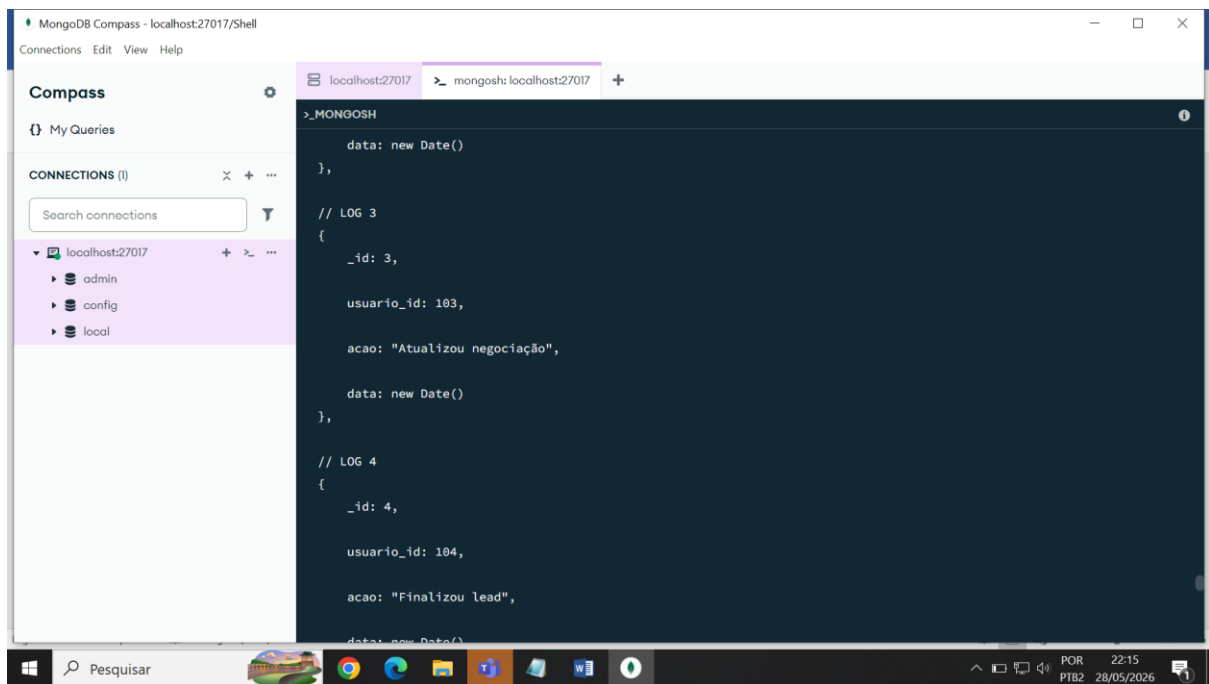
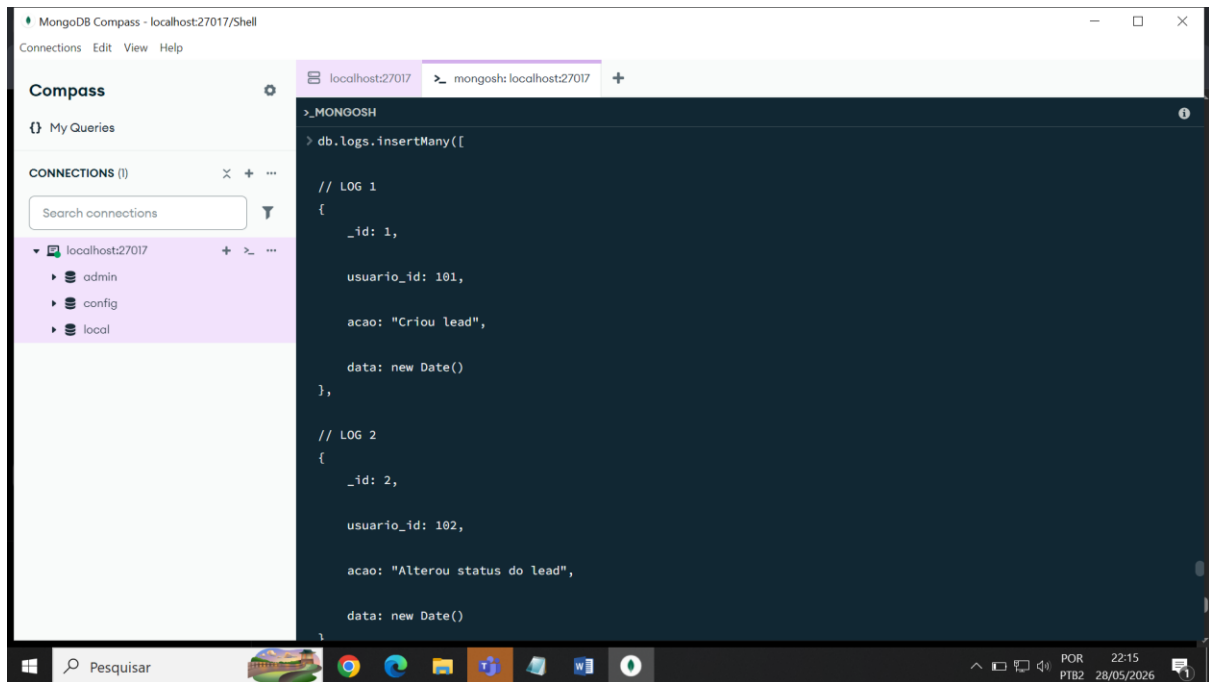
db.logs — Logs do Sistema

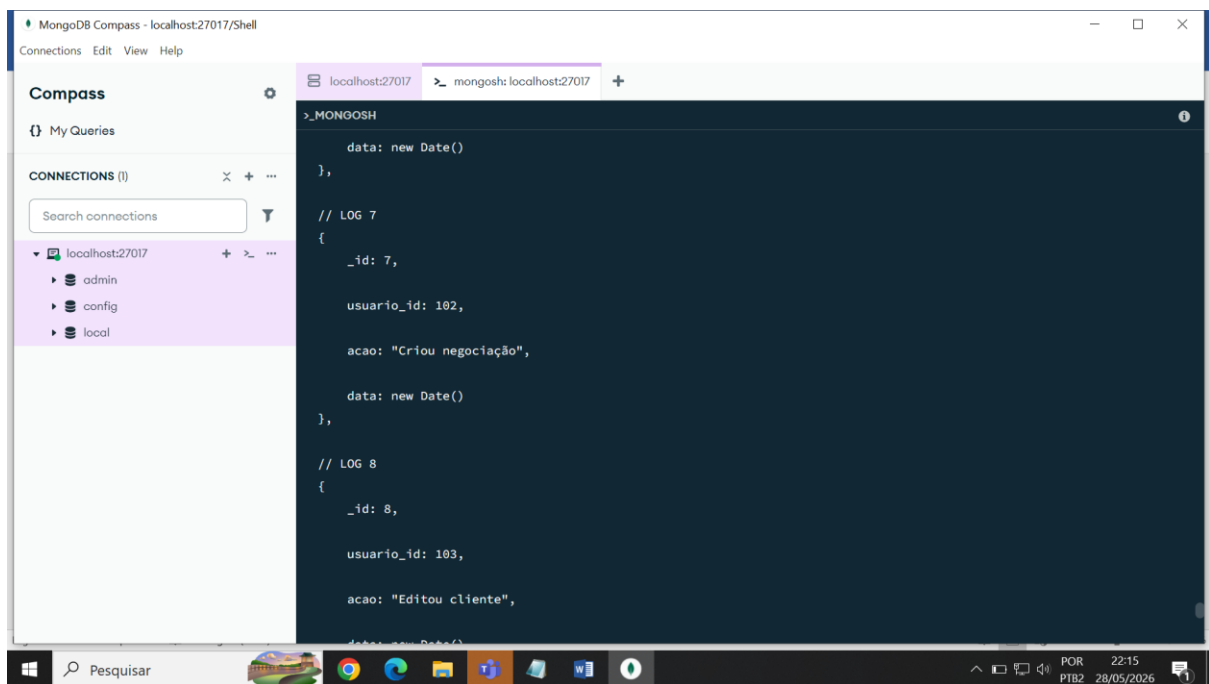
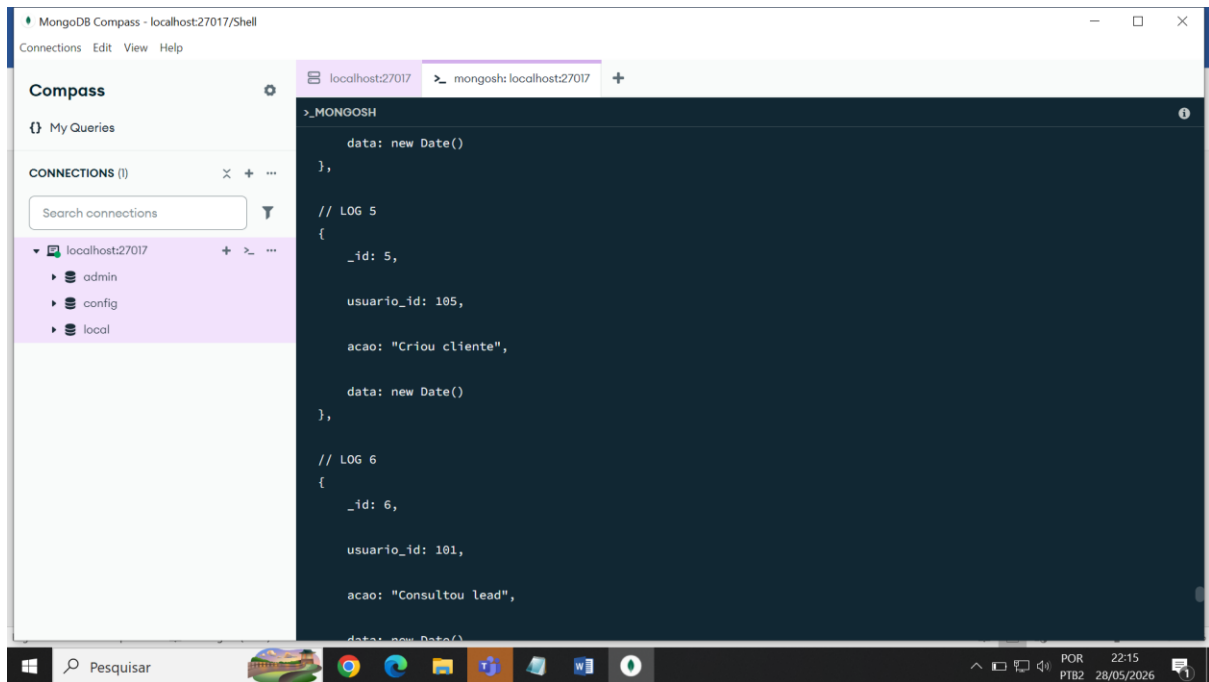
A collection logs foi criada para registrar todas as ações realizadas pelos usuários no sistema, garantindo rastreabilidade e controle de auditoria. Cada entrada corresponde a uma ação específica, como criação ou atualização de registros.

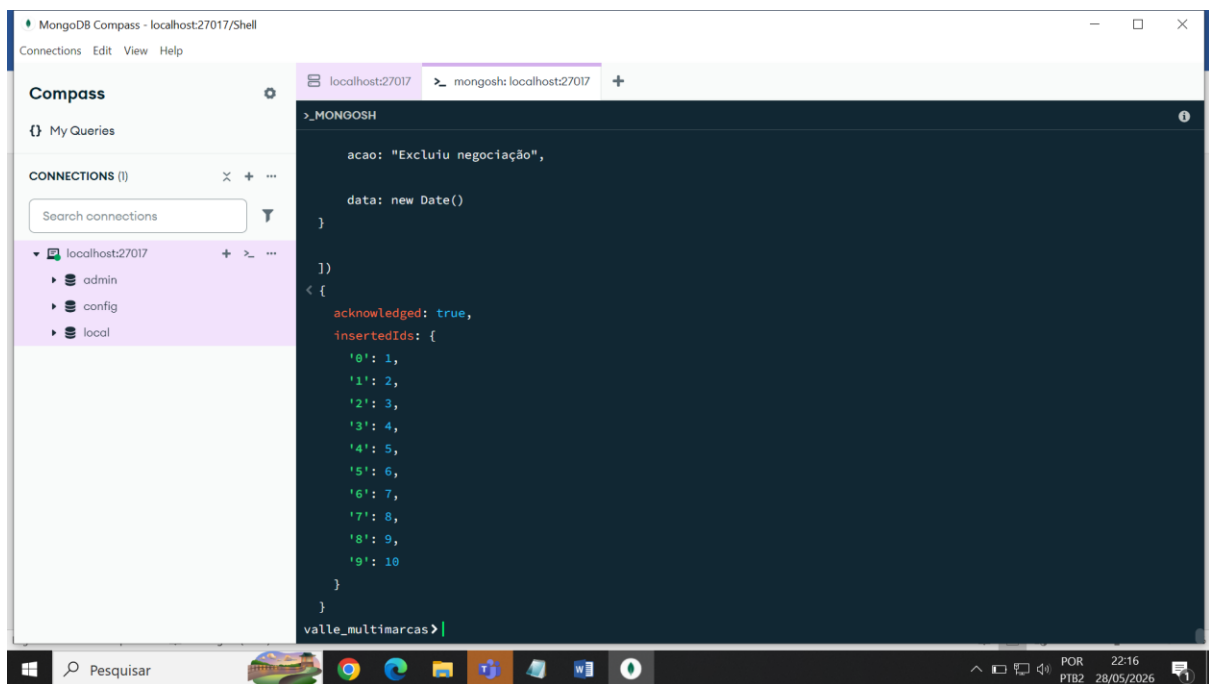
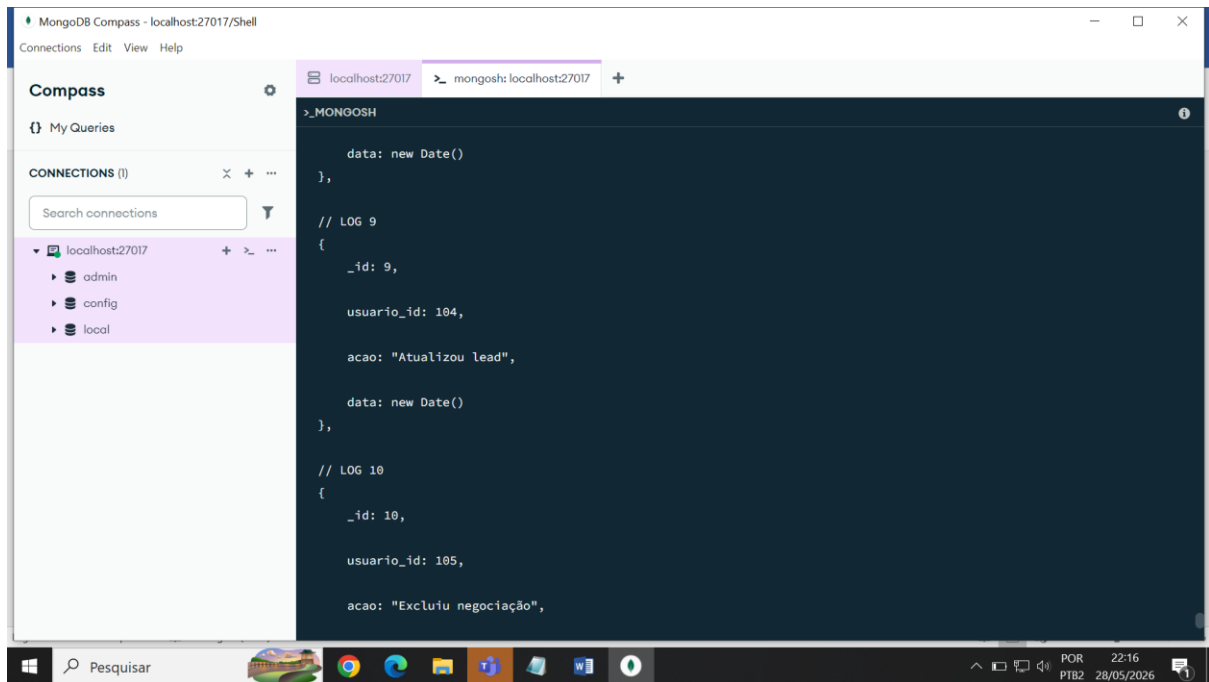
Campos de cada documento:

- usuario_id — referência ao usuário que realizou a ação
- acao — descrição da ação executada
- data — data e hora do evento

```
{ _id: 1,  usuario_id: 101, acao: 'Criou lead',          data: ISODate(...) }
{ _id: 2,  usuario_id: 102, acao: 'Alterou status do lead', data: ISODate(...) }
{
}
{ _id: 3,  usuario_id: 103, acao: 'Atualizou negociação', data: ISODate(...) }
{ _id: 7,  usuario_id: 102, acao: 'Criou negociação',      data: ISODate(...) }
{ _id: 10, usuario_id: 105, acao: 'Excluiu negociação',    data: ISODate(...) }
```







Consultas

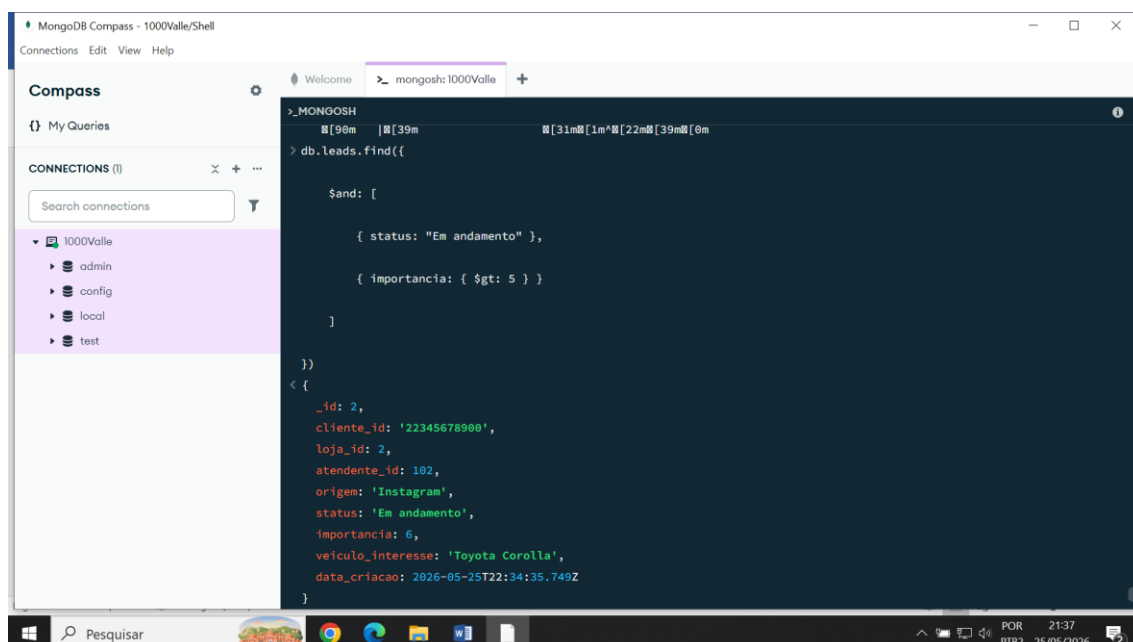
As consultas desenvolvidas permitiram aplicar operadores importantes do MongoDB para filtragem e análise das informações armazenadas. Foram utilizados operadores lógicos, projeções, ordenações e paginação de resultados, demonstrando a flexibilidade do MongoDB na manipulação dos dados.

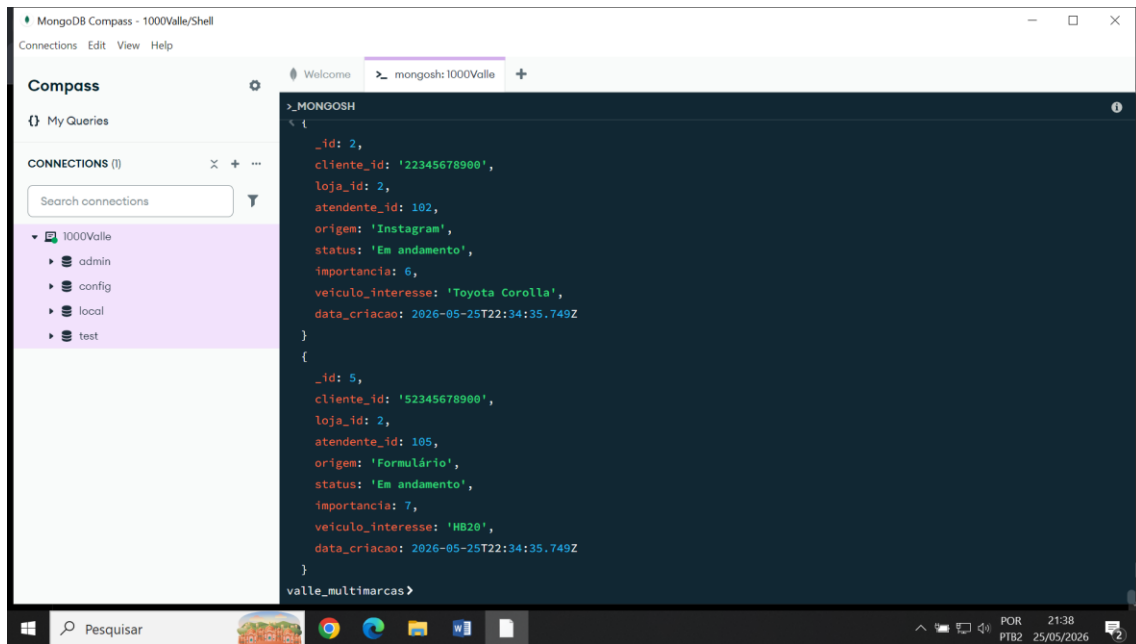
Operador \$and

O operador \$and combina múltiplas condições na mesma consulta, retornando apenas os documentos que satisfazem simultaneamente todos os critérios definidos. Isso torna a filtragem mais específica e organizada.

Exemplo — busca leads com status "Em andamento" e importância maior que 5:

```
db.leads.find({
  $and: [
    { status: 'Em andamento' },
    { importancia: { $gt: 5 } }
  ]
})
```



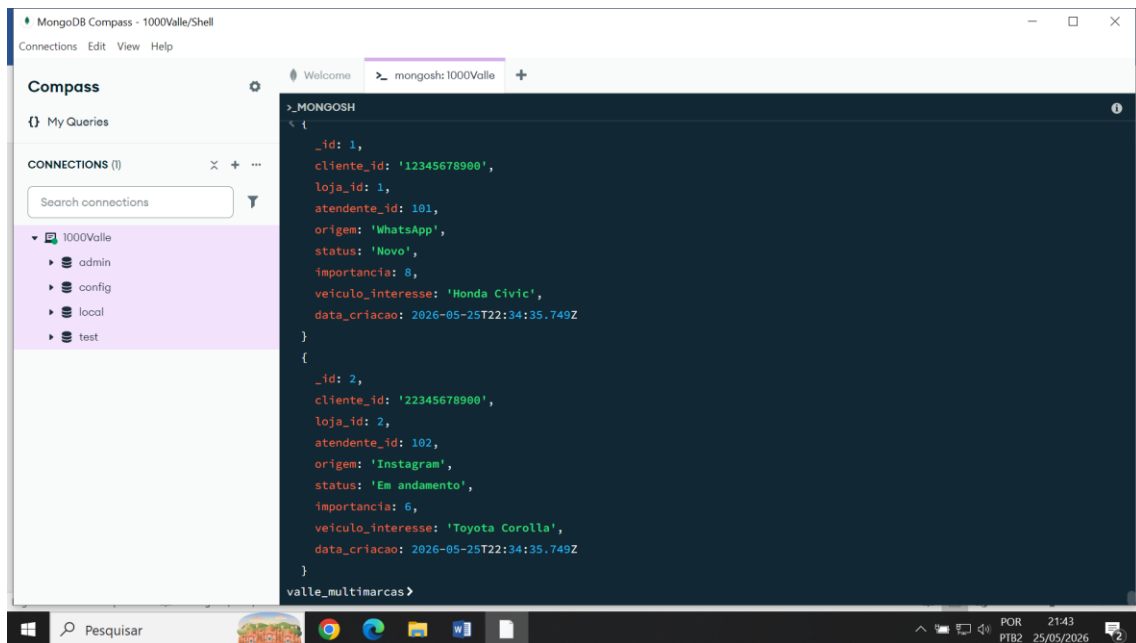
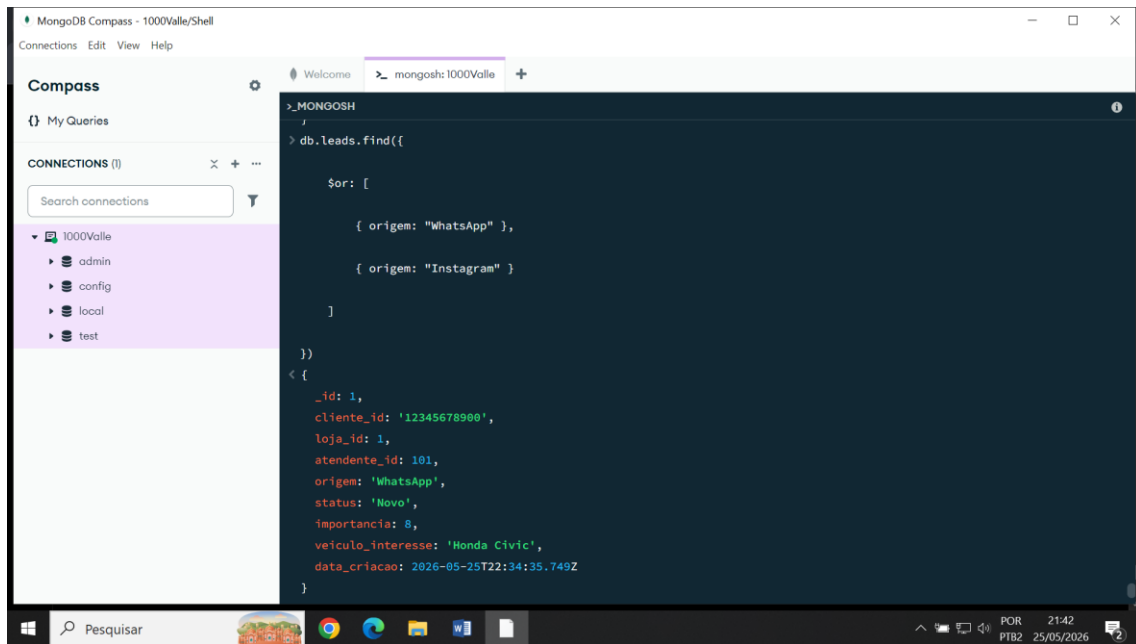


Operador \$or

O operador \$or realiza consultas flexíveis, retornando documentos que atendam pelo menos uma das condições definidas. É útil para localizar registros com diferentes características em uma única consulta.

Exemplo — busca leads originados do WhatsApp ou do Instagram:

```
db.leads.find({
  $or: [
    { origem: 'WhatsApp' },
    { origem: 'Instagram' }
  ]
})
```



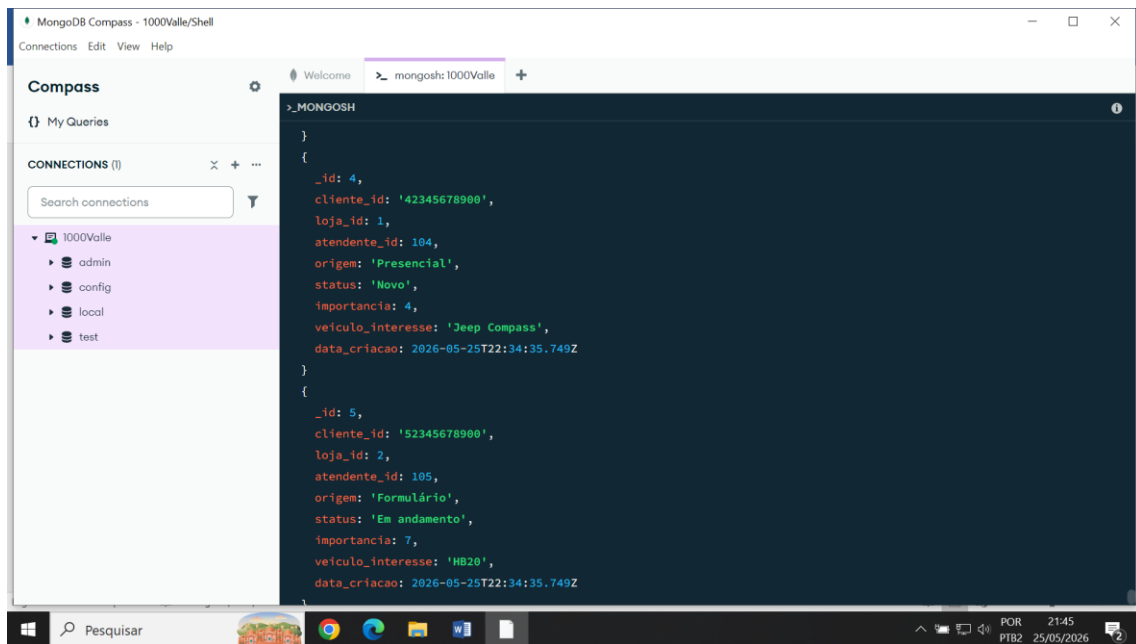
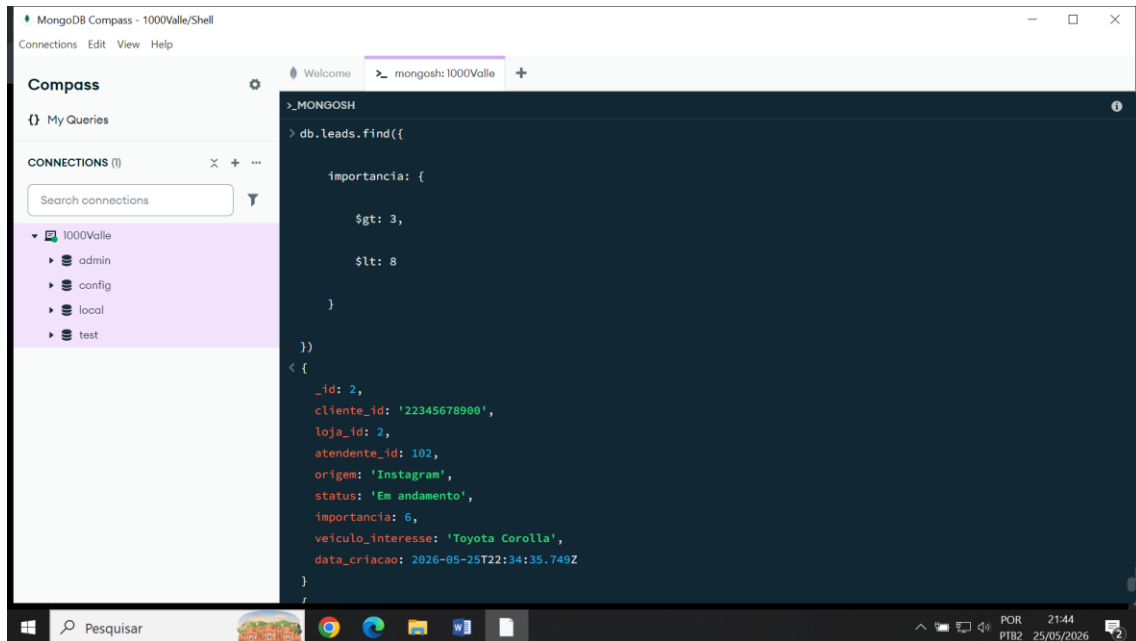
Operadores \$gt e \$lt

Os operadores \$gt (greater than — maior que) e \$lt (less than — menor que) realizam filtros comparativos em campos numéricos ou de data, permitindo localizar documentos dentro de um intervalo de valores.

Exemplo — busca leads com importância entre 3 e 8 (exclusive):

```
db.leads.find({
  importancia: {
    $gt: 3,
```

```
    $lt: 8
  }
})
```

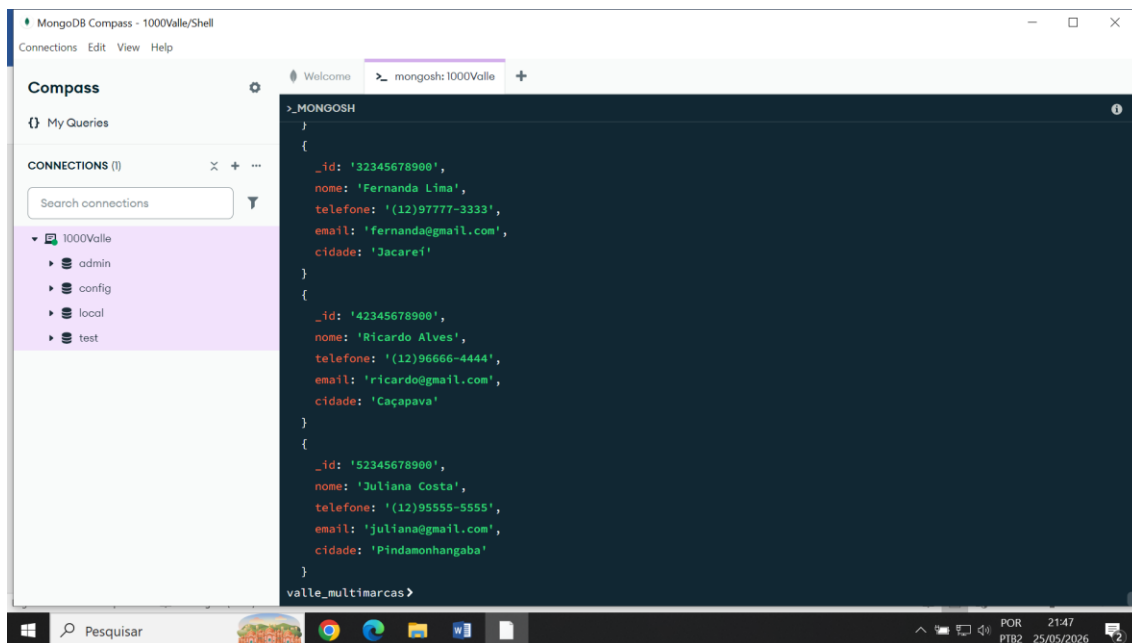
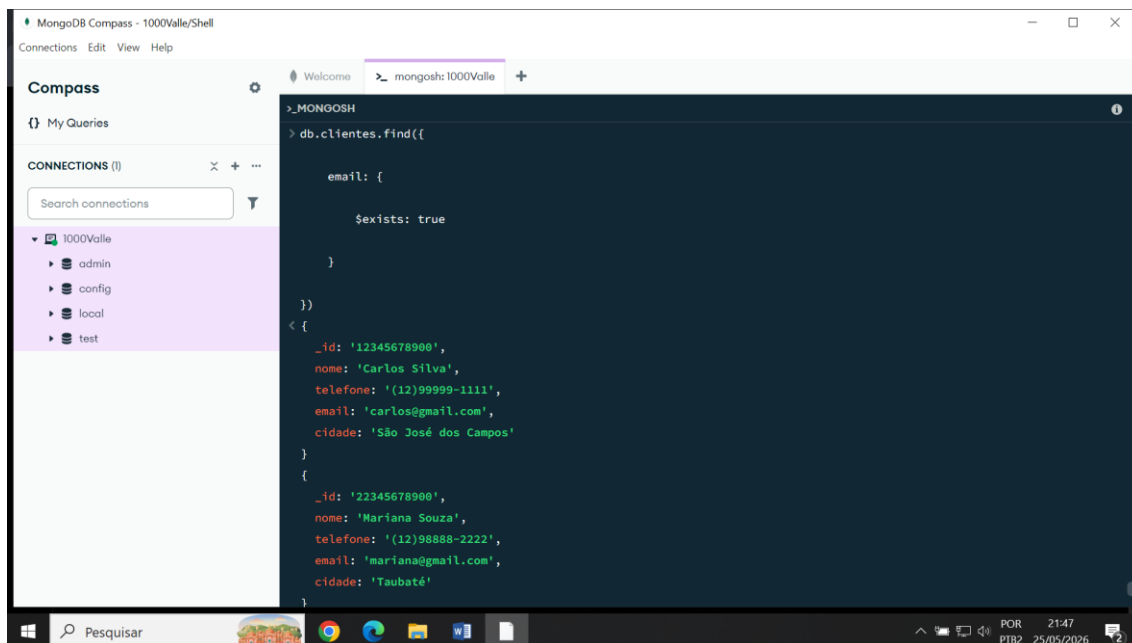


Operador \$exists

O operador `$exists` verifica a presença (ou ausência) de um campo específico nos documentos de uma collection. É muito útil para identificar registros com dados incompletos ou para validar a estrutura dos documentos.

Exemplo — busca todos os clientes que possuem o campo email cadastrado:

```
db.clientes.find({
  email: { $exists: true }
})
```

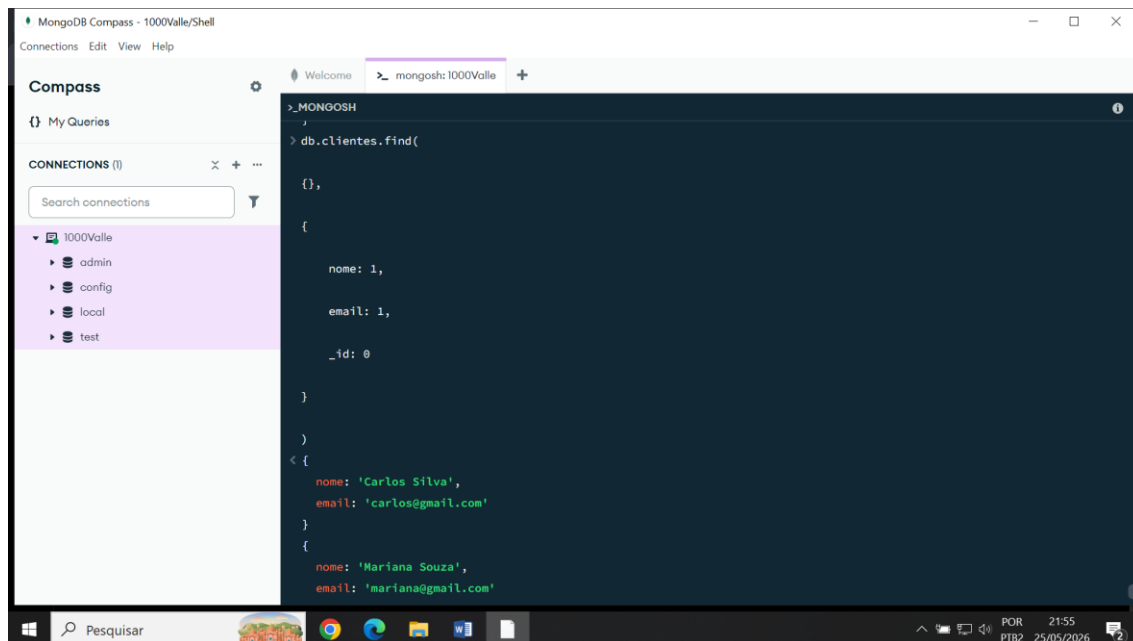


Projeção de Campos

A projeção permite selecionar apenas os campos necessários nos resultados de uma consulta, reduzindo a quantidade de dados retornados, melhorando o desempenho e facilitando a leitura das informações relevantes.

Exemplo — retorna apenas nome e email dos clientes, sem exibir o `_id`:

```
db.clientes.find(
  {},
  { nome: 1, email: 1, _id: 0 }
)
```

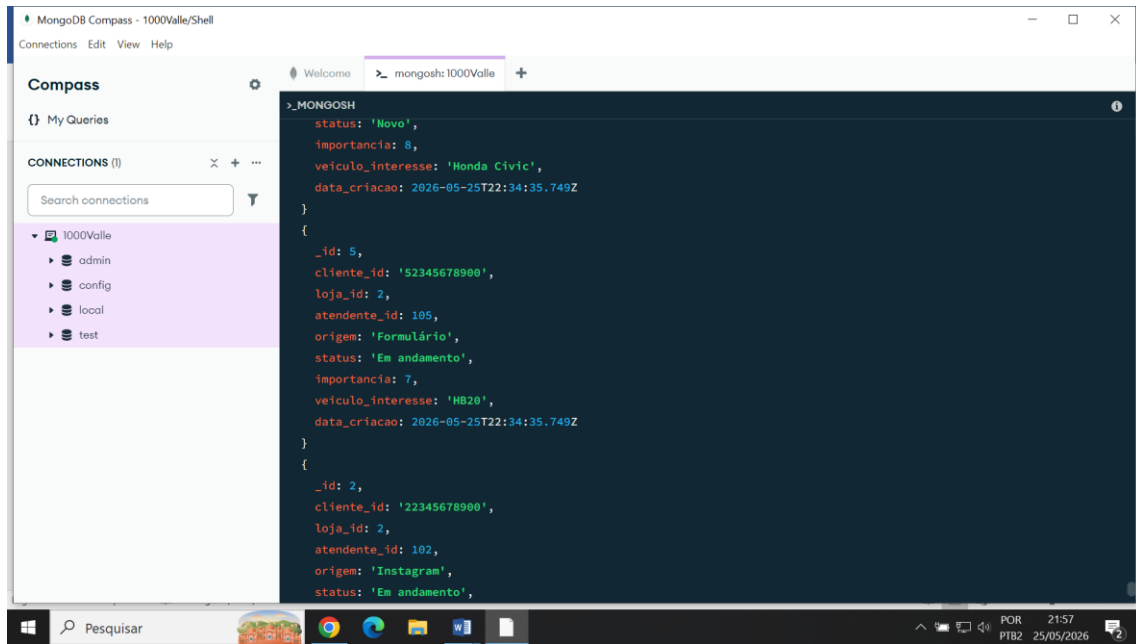
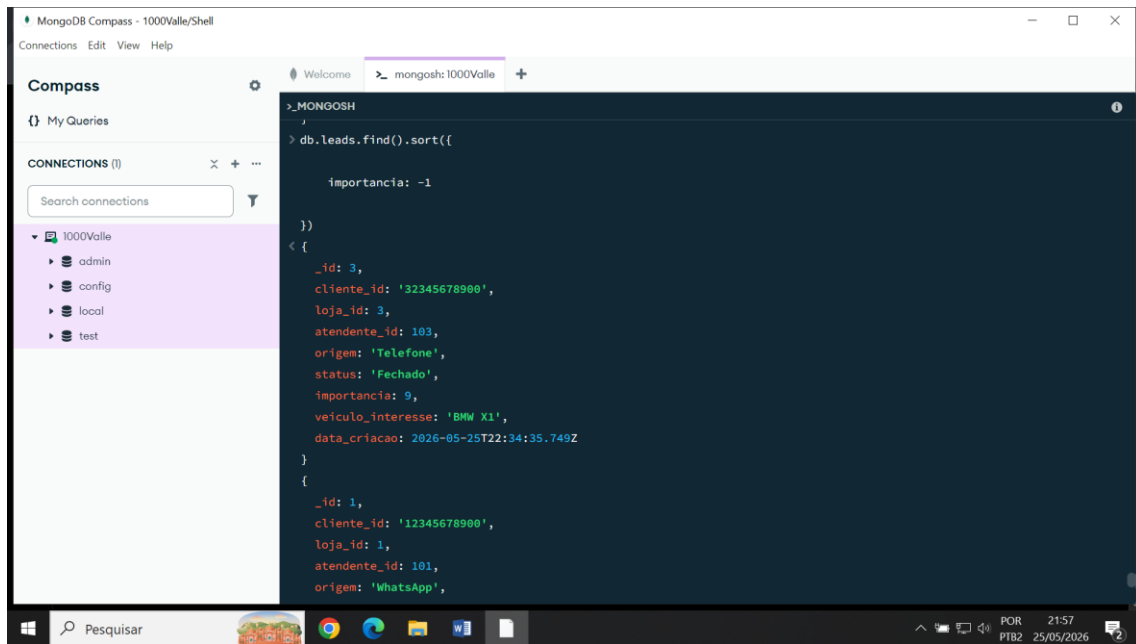


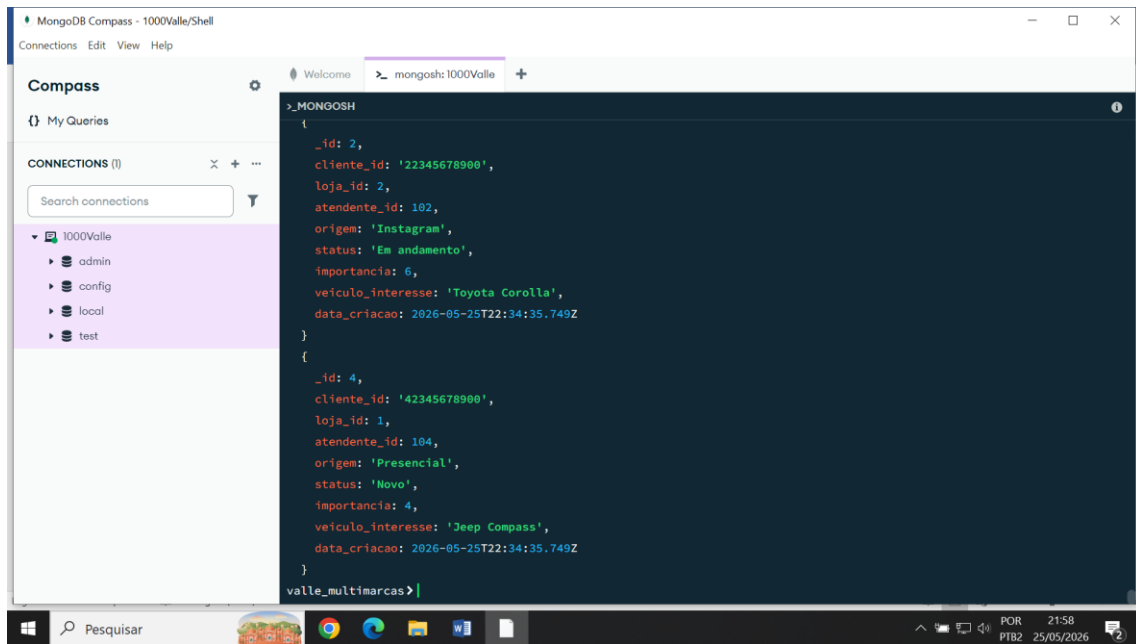
Ordenação com `sort()`

O método `sort()` organiza os resultados de uma consulta em ordem crescente (1) ou decrescente (-1). Facilita a visualização de registros por prioridade, datas recentes ou qualquer outro critério de ordenação.

Exemplo — ordena todos os leads do mais importante para o menos importante:

```
db.leads.find().sort({
  importancia: -1 // -1 = decrescente | 1 = crescente
})
```



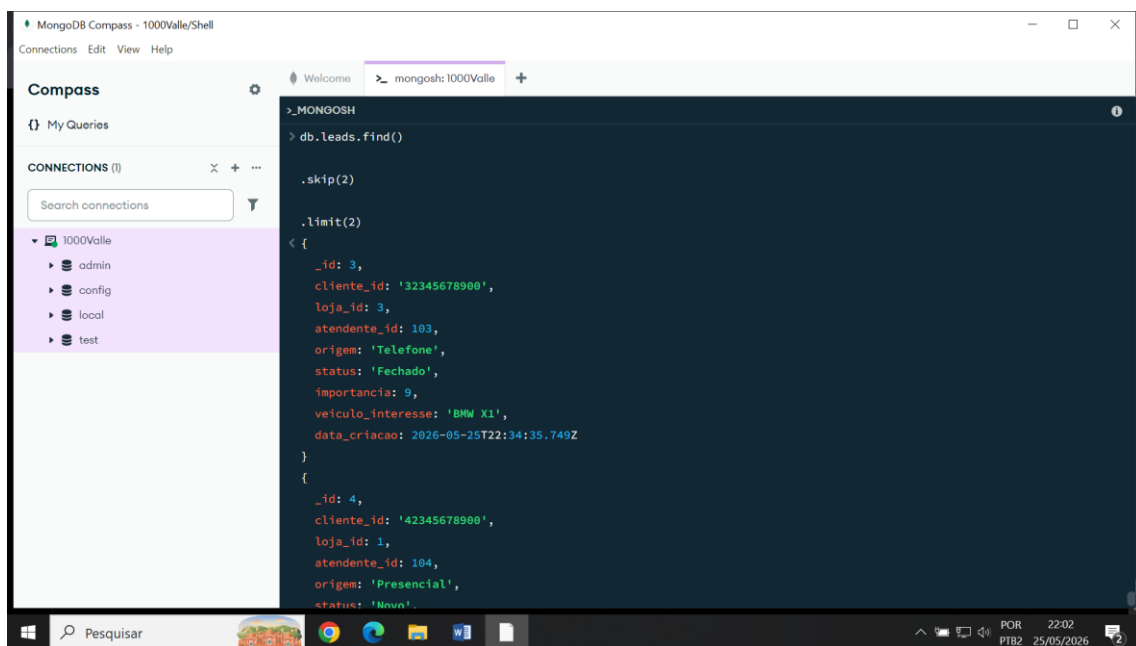


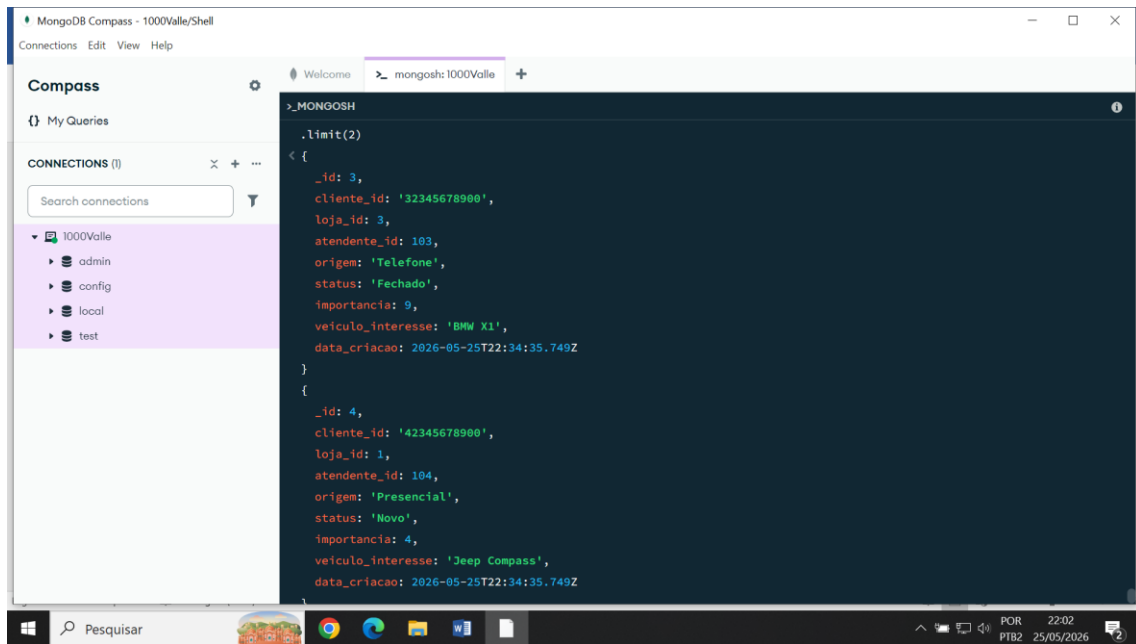
Paginação com skip() e limit()

A paginação divide os resultados em blocos menores, melhorando o desempenho e a experiência de navegação em grandes volumes de dados. O método skip() pula um número de documentos e o limit() define quantos serão exibidos.

Exemplo — pula os 2 primeiros resultados e exibe os próximos 2 (página 2):

```
db.leads.find()
  .skip(2)    // pula 2 documentos
  .limit(2)   // exibe 2 documentos
```





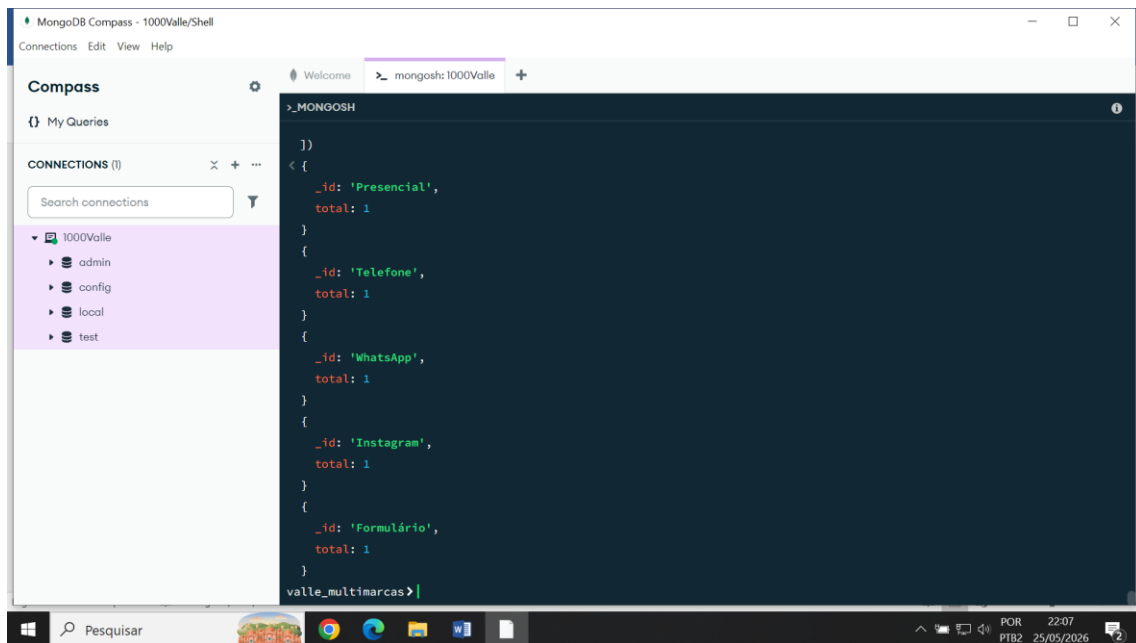
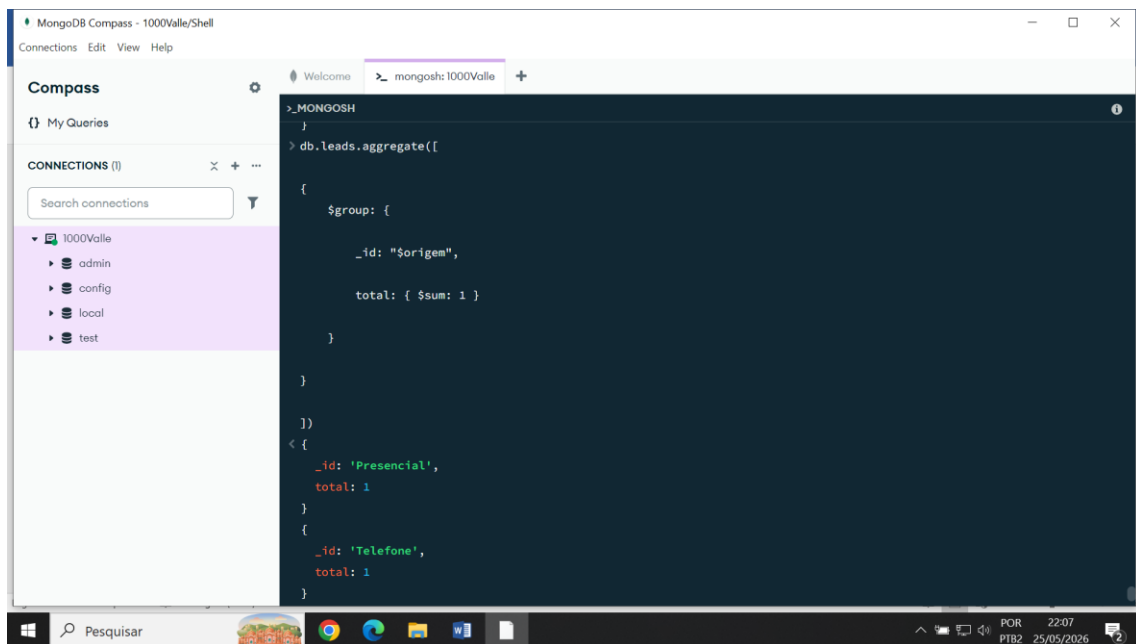
Aggregations

As aggregations foram utilizadas para gerar indicadores gerenciais do sistema, permitindo visualizar informações estratégicas sobre os leads cadastrados. Através das pipelines foi possível identificar quantidade de leads por origem, status, atendente e importância, auxiliando na análise do desempenho comercial.

Aggregation 1 — Leads por Origem

Agrupar os leads de acordo com o canal pelo qual o cliente entrou em contato (WhatsApp, Instagram, Telefone, Presencial, Formulário) e contar quantos registros existem em cada canal. Permite identificar as fontes de captação mais eficientes para a empresa.

```
db.leads.aggregate([
  {
    $group: {
      _id: '$origem',
      total: { $sum: 1 }
    }
  }
])
```

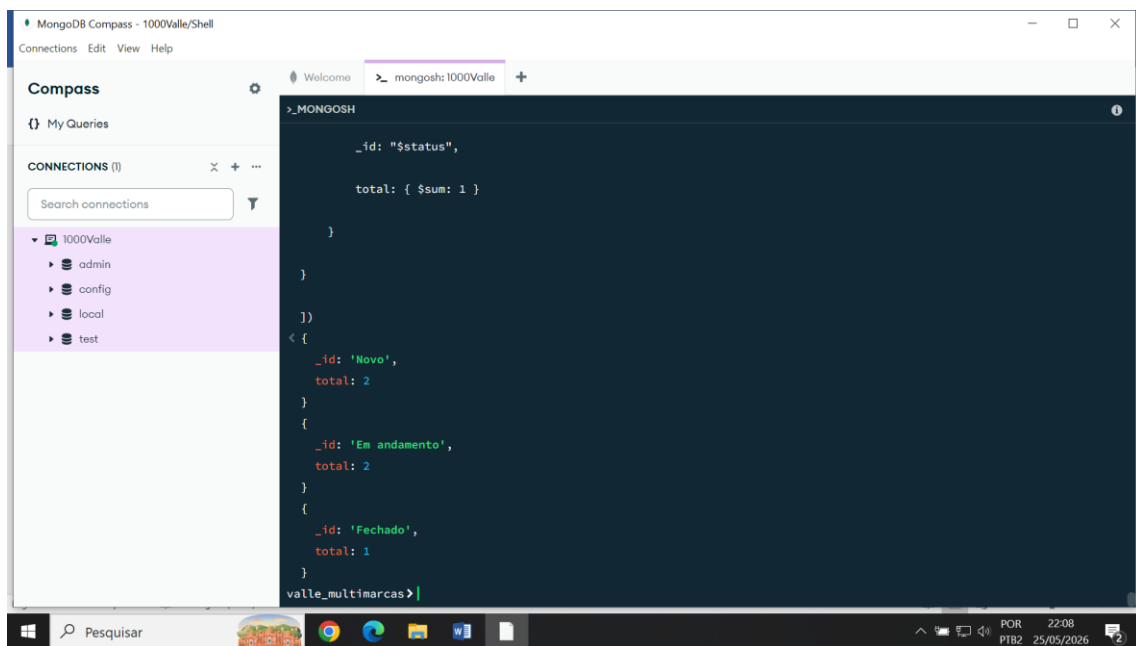
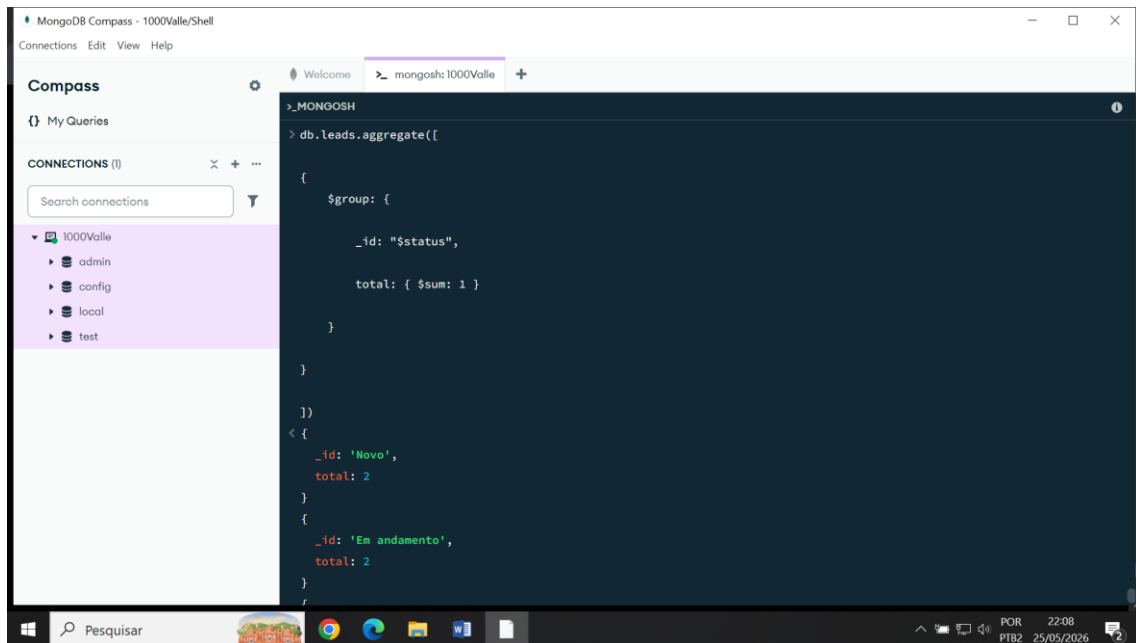



Aggregation 2 — Leads por Status

Agrupar os leads conforme o status atual da negociação (Novo, Em andamento, Fechado) e contabilizar a quantidade em cada etapa. Fornece uma visão organizada e estratégica do andamento do processo comercial.

```
db.leads.aggregate([
  {
    $group: {
      _id: '$status',
      total: { $sum: 1 }
    }
  }
])
```

])



Aggregation 3 — Leads por Importância

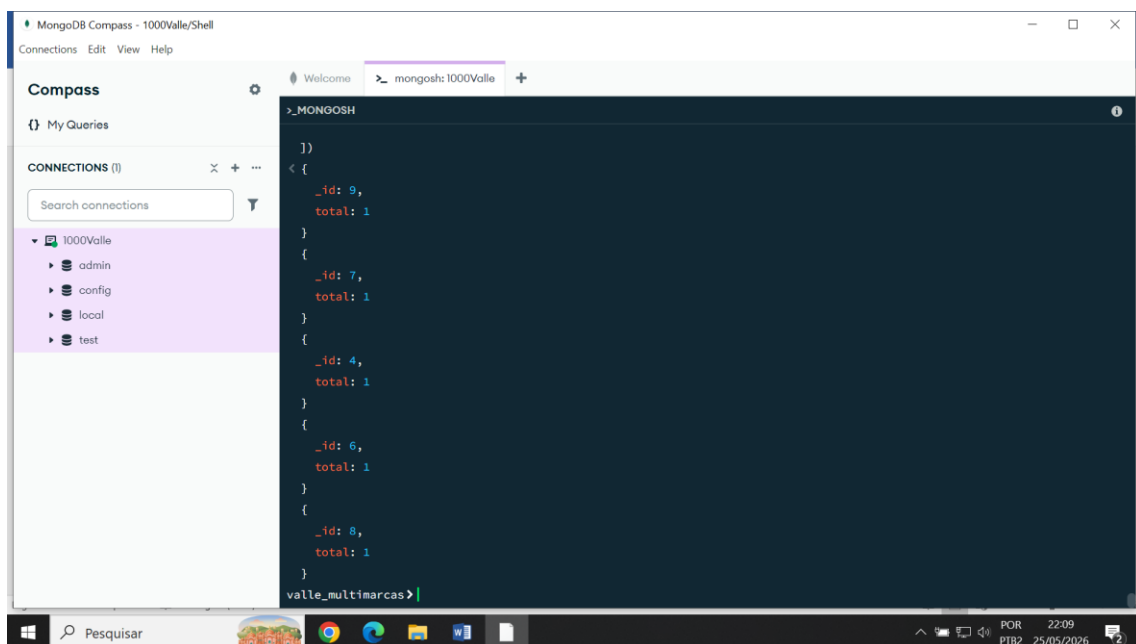
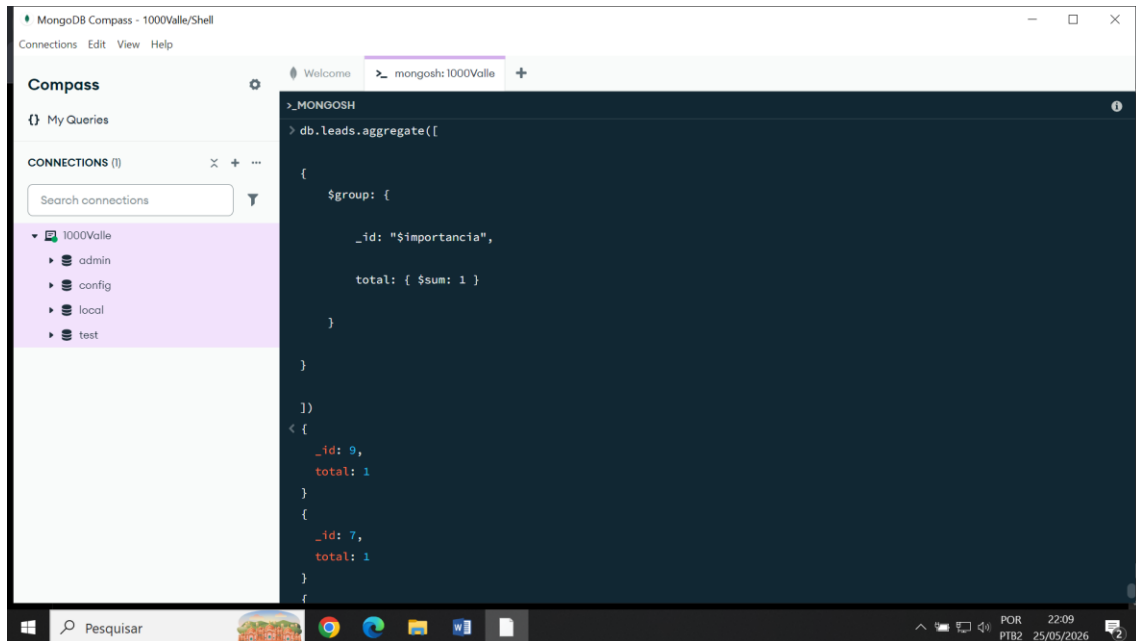
Agrupar os leads de acordo com o nível de importância atribuído (escala de 1 a 10), contabilizando quantos leads existem em cada categoria de prioridade. Auxilia na organização e análise estratégica dos atendimentos.

```
db.leads.aggregate([
  {
    $group: {
```

```

        _id: '$importancia',
        total: { $sum: 1 }
    }
}
])

```



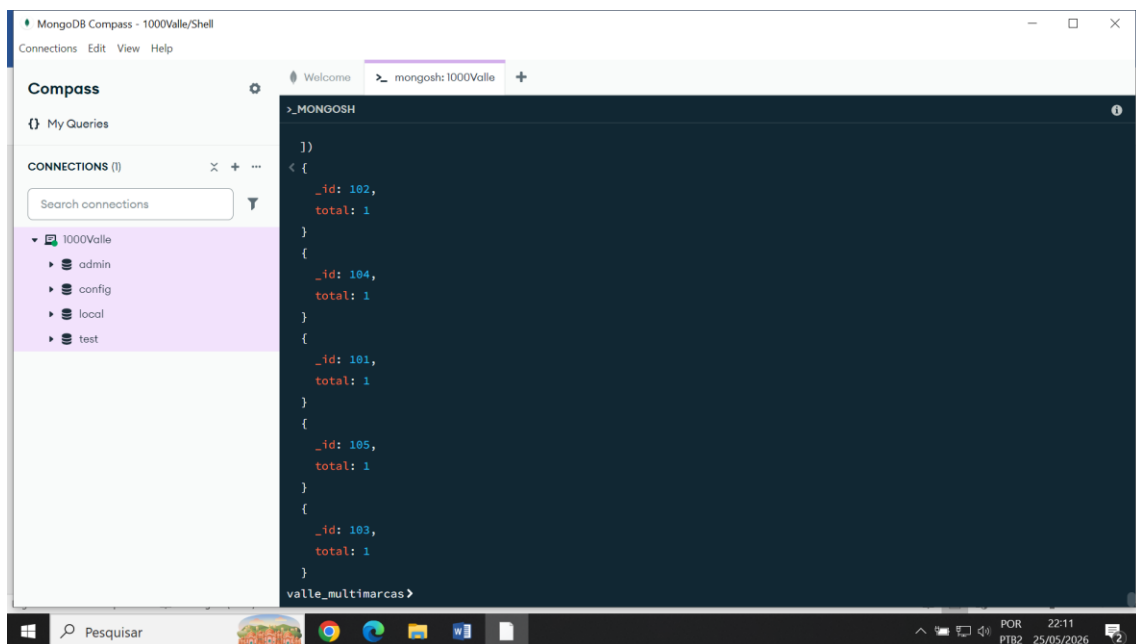
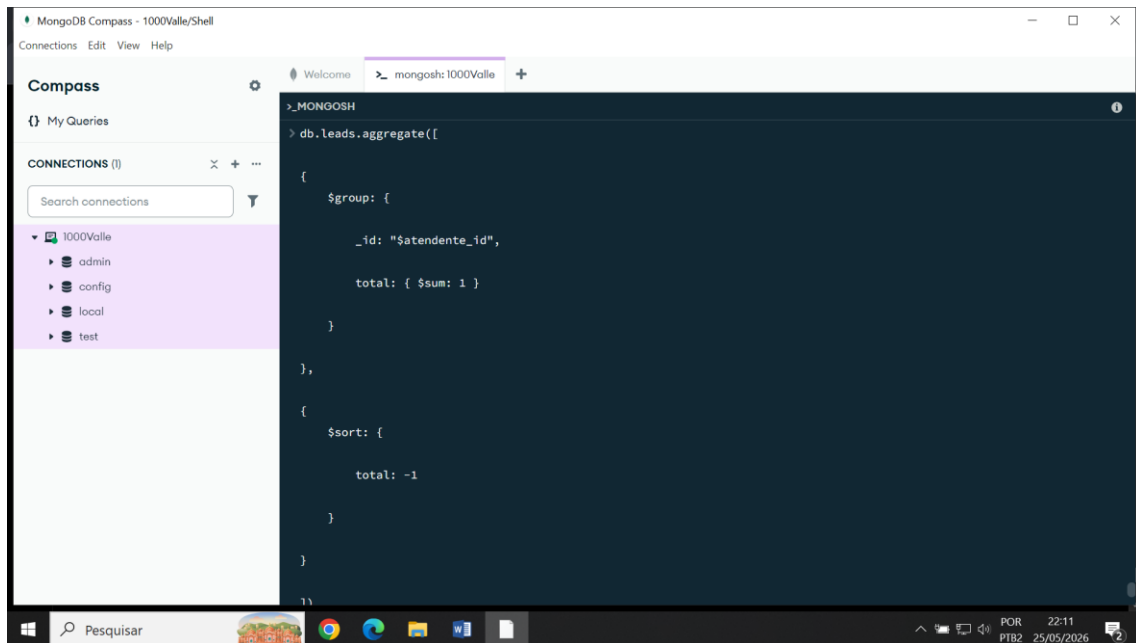
Aggregation 4 — Leads por Atendente

Agrupar os leads conforme o atendente responsável e ordenar o resultado em ordem decrescente de quantidade. Permite analisar a distribuição dos atendimentos entre os colaboradores, identificando desequilíbrios de carga.

```

db.leads.aggregate([
  {
    $group: {
      _id: '$atendente_id',
      total: { $sum: 1 }
    }
  },
  {
    $sort: { total: -1 }    // ordena do maior para o menor
  }
])

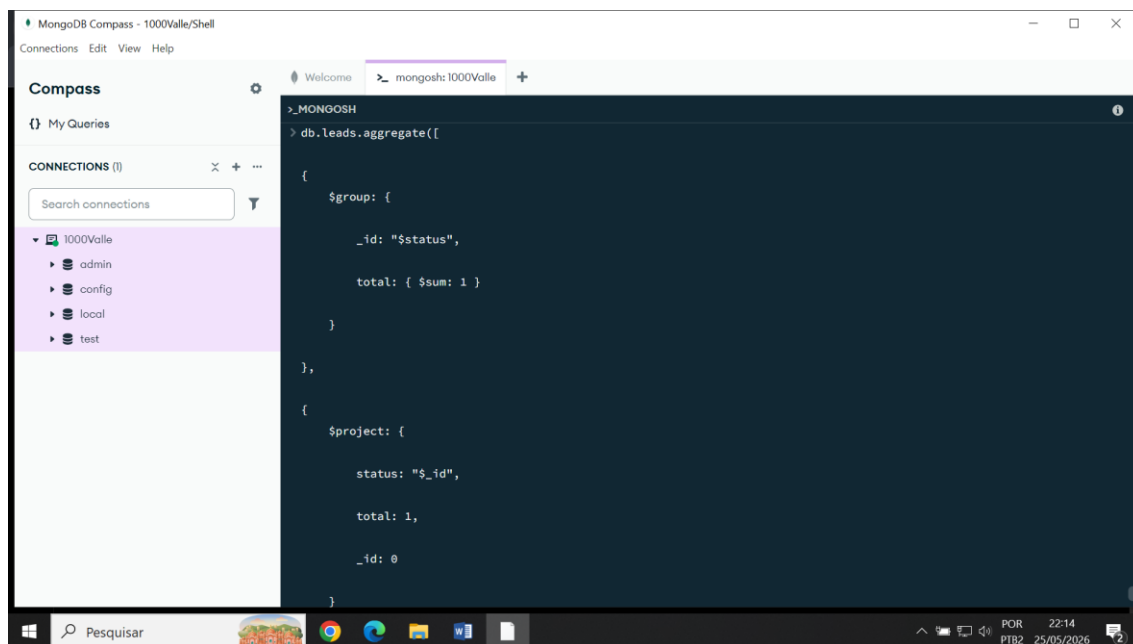
```

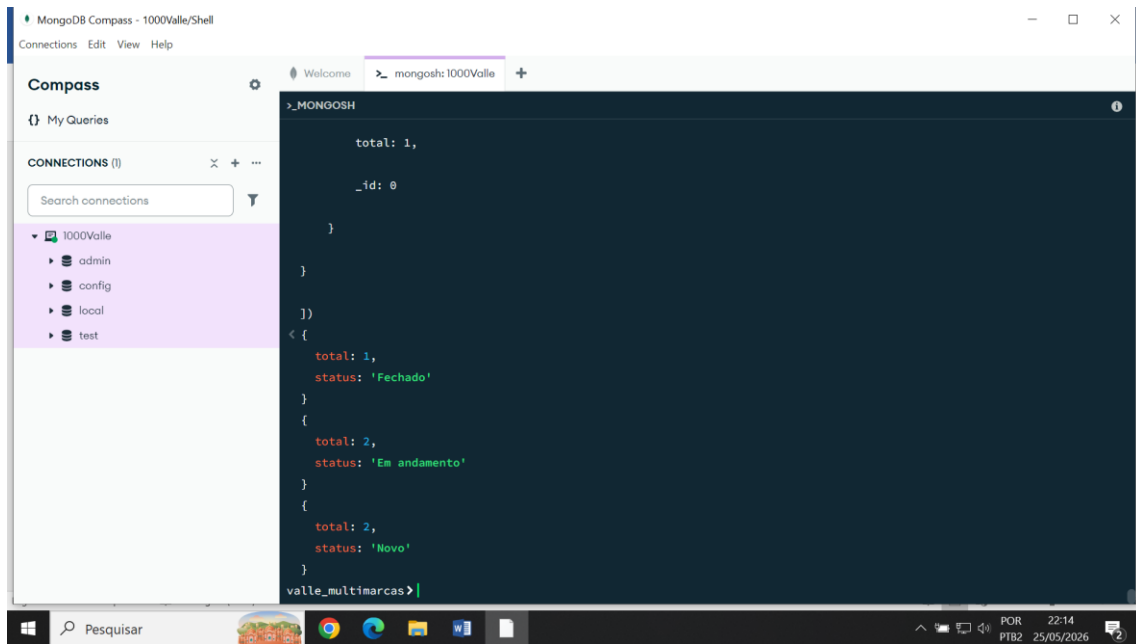
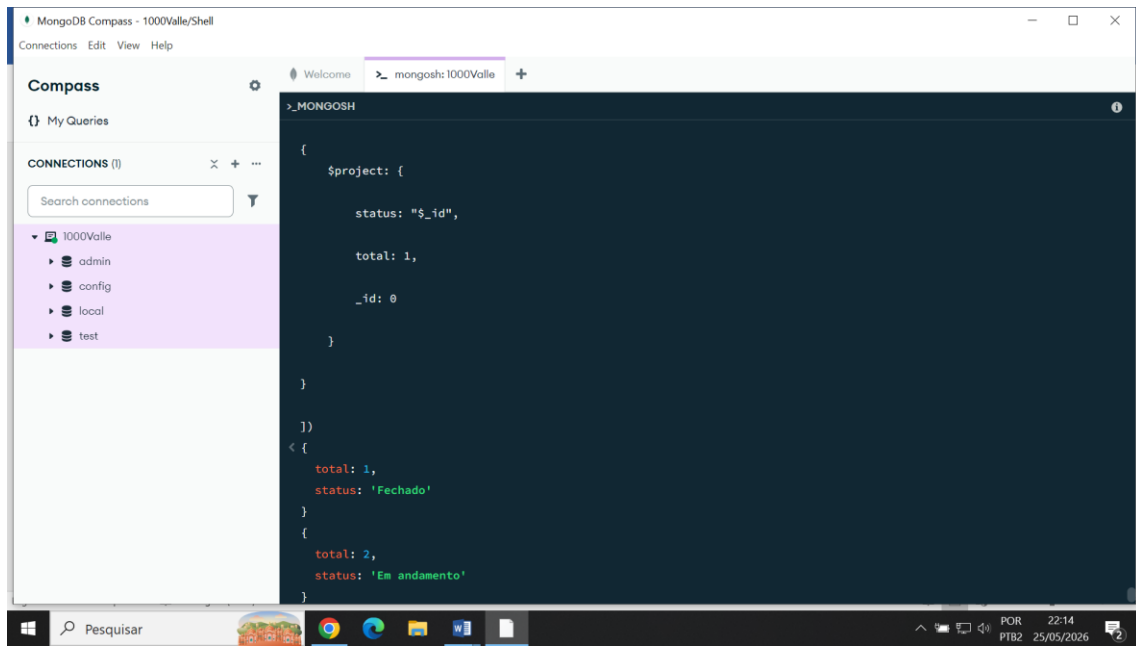


Aggregation 5 — Taxa de Conversão

Agrupar os leads pelo campo status e utiliza o estágio \$project para renomear o campo de saída, tornando o resultado mais legível. Permite identificar quantos leads foram convertidos (Fechado) em relação ao total de negociações registradas no sistema.

```
db.leads.aggregate([
  {
    $group: {
      _id: '$status',
      total: { $sum: 1 }
    }
  },
  {
    $project: {
      status: '$_id',
      total: 1,
      _id: 0
    }
  }
])
```





Resumo dos Operadores e Métodos

A tabela abaixo resume todos os operadores e métodos utilizados no projeto, com sua função e um exemplo prático de aplicação no sistema de gestão de leads.

Operador / Método	Função	Exemplo Prático
\$and	Todas as condições verdadeiras	status = "NOVO" e vendedor = "Carlos"
\$or	Pelo menos uma condição verdadeira	status = "NOVO" ou vendedor = "Carlos"
\$gt	Valores maiores que outro valor	importância > 5
\$lt	Valores menores que outro valor	valor < 1000
\$exists	Verifica se um campo existe	telefone existe
Projection	Mostra apenas campos específicos	mostrar nome e e-mail
sort()	Organiza resultados	maior → menor
skip()	Pula registros	pula 5 documentos
limit()	Limita quantidade exibida	mostra apenas 5
aggregate()	Processa e analisa dados	relatórios e agrupamentos
\$group	Agrupar documentos	leads por status
\$sum	Conta ou soma valores	quantidade de leads

Conclusão

O desenvolvimento do banco de dados não relacional utilizando MongoDB permitiu representar de forma eficiente o sistema de gestão de leads da empresa 1000 Valle Multimarcas.

Durante o projeto foram criadas seis collections — clientes, lojas, usuários, leads, negociações e logs — possibilitando melhor organização dos dados e controle completo das informações comerciais do sistema.

Foram aplicados conceitos importantes da modelagem não relacional, como embedding (histórico de negociações dentro do próprio documento) e referencing via ObjectId (relacionamento entre collections), permitindo armazenar históricos de negociações de forma prática e eficiente.

As consultas com operadores do MongoDB (\$and, \$or, \$gt, \$lt, \$exists, projeção, sort, skip, limit) e as cinco aggregations construídas geraram indicadores gerenciais relevantes, como análise de leads por origem, status, atendente e importância, apoiando a tomada de decisão comercial da empresa.

Por fim, o projeto consolidou o aprendizado prático sobre bancos de dados não relacionais, demonstrando as vantagens do MongoDB em relação à flexibilidade estrutural, facilidade de manipulação dos dados e sua excelente adaptação para sistemas modernos e escaláveis.